Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра электронных вычислительных машин

Отчет
по учебной(ознакомительной) практике
на тему

РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ ДЛЯ ИГРЫ В ШАШКИ

Студент:
гр. 458306 Тукайло М. А.

Руководитель:
ассистент кафедры ЭВМ
Тарасюк И. С.

МИНСК 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 ОБЗОР ЛИТЕРАТУРЫ.....4
1.1 Шашки.....4
1.2 Правила игры в шашки.....4
1.3 Обзор существующих аналогов.....5
1.4 Обзор технологий для реализации.....6
1.5 Обзор игровых алгоритмов.....7
2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....8
2.1 Реализация алгоритма хода шашки.....8
2.2 Реализация алгоритма хода дамки.....10
2.3 Реализация ИИ на основе «жадного» алгоритма.....13
3 ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ.....17
ЗАКЛЮЧЕНИЕ.....20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....21
ПРИЛОЖЕНИЕ А.....22
ПРИЛОЖЕНИЕ Б.....23

ВВЕДЕНИЕ

В современном мире информационные технологии играют ключевую роль и проникают практически во все сферы человеческой деятельности – от промышленности и науки до сферы досуга и образования. Особенно заметным стало их влияние на область цифровых развлечений, где стремительное развитие компьютерных и мобильных игр открыло широкие возможности как для пользователей, так и для разработчиков. Одним из популярных и востребованных жанров по – прежнему остаются настольные игры, в том числе хорошо знакомая с детства игра в шашки. Эта классическая логическая игра, несмотря на свою кажущуюся простоту, обладает глубоким стратегическим содержанием и способствует развитию аналитического мышления.

Целью данной учебной (ознакомительной) практики является разработка приложения для игры в шашки на языке программирования C.

В ходе работы планируется спроектировать структуру данных для представления игрового поля и фигур, реализовать базовые игровые механики (перемещение, взятие, правила превращения в дамку), а также предусмотреть простой пользовательский интерфейс для взаимодействия с программой в консольном режиме. Кроме того, будут рассмотрены алгоритмы проверки корректности ходов и определения окончания игры.

Реализация данной программы позволит углубить практические навыки работы с языком C и повысить понимание логики построения игровых приложений.

1 ОБЗОР ЛИТЕРАТУРЫ

1.1 Шашки

Шашки – это настольная логическая игра для двух игроков, имеющая многовековую историю и широко распространенная во всем мире. Игра проходит на квадратной доске размером 8×8 клеток, аналогичной шахматной, но используется только темная часть клеток. Каждый игрок управляет набором фигур – шашек, которые перемещаются по диагонали и могут «бить» (перепрыгивать) фигуры противника, убирая их с поля.

Цель игры заключается в том, чтобы захватить все шашки соперника. При достижении последней горизонтали противника обычная шашка превращается в дамку – фигуру с расширенными возможностями передвижениях [1].

Несмотря на внешнюю простоту и ограниченный набор правил, игра в шашки требует от игрока развитого логического мышления, способности к стратегическому планированию и умения просчитывать возможные ходы на несколько шагов вперед. Каждое действие на игровом поле может повлиять на исход партии, поэтому важно не только реагировать на текущую ситуацию, но и предвидеть потенциальные ходы соперника.

Благодаря сочетанию доступности и глубины, шашки продолжают оставаться популярными уже на протяжении многих десятилетий. Эта игра востребована как в своем классическом настольном виде, так и в современных цифровых реализациях – в виде компьютерных программ и мобильных приложений. Интуитивно понятные правила делают ее доступной для новичков, а стратегическая насыщенность позволяет сохранять интерес даже у опытных игроков.

1.2 Основные правила игры

В классической игре в шашки принимают участие два игрока, каждый из которых управляет своей группой фигур. На старте партии каждому игроку предоставляется по 12 шашек, которые размещаются на темных клетках первых трех горизонтальных рядов доски, расположенных ближе к нему. Такое начальное расположение обеспечивает симметричность поля и равные условия для обеих сторон.

Ходы в игре выполняются по очереди: один игрок делает свой ход, затем – второй. Передвижение фигур осуществляется строго по диагонали, и на начальном этапе игры простые (или как их еще называют – обычные) шашки могут двигаться только вперед, на одну клетку по диагонали. Это означает, что шашка не может перепрыгивать через фигуры или ходить назад, если она еще не достигла статуса дамки.

Одним из ключевых элементов игрового процесса является взятие

шашки соперника. Если во время хода открывается возможность побить фигуру противника, игрок может выполнить такой ударный ход, перепрыгивая через вражескую шашку. Однако, в отличие от некоторых других версий шашек, здесь взятие не является обязательным – игрок сам решает, воспользоваться ли возможностью или предпочесть иной ход.

Особое значение в шашках имеет превращение обычной шашки в дамку. Это происходит, когда шашка достигает самой дальней горизонтали на стороне противника. Получив статус дамки, фигура меняет свои игровые свойства: теперь она может перемещаться по диагонали на любое расстояние в любую сторону – вперед и назад, а также совершать взятия на дальних расстояниях, что делает ее особенно ценной в стратегическом плане.

Таким образом, правила движения и взятия фигур в шашках формируют интересный и продуманный игровой процесс, в котором важны как тактические решения на каждом ходе, так и стратегическое планирование на протяжении всей партии. [2]

1.3 Обзор существующих аналогов

Существует множество реализаций игры в шашки в виде компьютерных программ, каждая из которых обладает своей уникальной архитектурой, различной степенью сложности, уровнем проработки логики и объемом реализованных функций. Эти программы варьируются от простейших учебных прототипов до полноценных графических приложений с продвинутым искусственным интеллектом и сетевым режимом.

Одним из характерных и наглядных примеров является консольное приложение с открытым исходным кодом под названием **Checkers.c**, размещенное на платформе **GitHub**. Эта программа представляет собой простую, но функциональную реализацию классических шашек с использованием языка программирования **C**. Ее особенностью является минималистичный подход: отсутствует графический интерфейс, а все взаимодействие с пользователем происходит исключительно через консоль. Благодаря этому код становится доступным для изучения даже начинающими программистами, что делает проект особенно ценным с образовательной точки зрения.

Программа Checkers.c реализует основные правила классической шашечной игры. Двое игроков поочередно делают ходы, управляя фигурами посредством ввода команд в текстовом режиме. Игровое поле отображается в виде таблицы из символов, где условными обозначениями представлены шашки и пустые клетки. Также предусмотрена логика проверки корректности совершаемых ходов, включая возможность взятия фигуры соперника. В дополнение к этому реализовано превращение обычной шашки в дамку при достижении ею противоположной стороны доски, что соответствует официальным правилам игры.

Несмотря на кажущуюся простоту, Checkers.c содержит все ключевые элементы, необходимые для функционирования базовой логической игры. Программа может служить отправной точкой для дальнейшего расширения и усложнения: ее можно дополнить системой очков, искусственным интеллектом, улучшенной проверкой правил, а также перенести в графическую среду. Код написан достаточно понятно и структурировано, что позволяет использовать его в учебных целях – например, при изучении структур данных, функций, обработки пользовательского ввода и простейших алгоритмов принятия решений.

Таким образом, данное приложение не только демонстрирует работу элементарной логики игры в шашки, но и служит отличным примером того, как можно реализовать пошаговую стратегию в виде простой программы на языке C. Благодаря открытости кода и отсутствию избыточной сложности, Checkers.c представляет собой удобную и полезную основу для разработки собственных игровых проектов, а также для практического освоения языка программирования и принципов построения игр.

1.4 Обзор технологий для реализации

При разработке консольных приложений на языке программирования C особое внимание уделяется организации пользовательского интерфейса, который, несмотря на свою текстовую природу, играет важнейшую роль в обеспечении комфортного взаимодействия пользователя с программой. Даже при отсутствии визуальных графических элементов, характерных для современных GUI–интерфейсов, консоль может быть достаточно информативной, визуально структурированной и удобной в использовании. Это достигается за счет применения различных приемов и инструментов, предоставляемых как стандартными, так и системными библиотеками.

Одной из ключевых библиотек, активно применяемых при разработке под операционную систему Windows, является `windows.h`. Данная библиотека предоставляет программисту доступ ко множеству функций низкоуровневого взаимодействия с компонентами операционной системы. В частности, с ее помощью можно реализовать задержку выполнения программы (что особенно важно для пошаговых интерфейсов), управлять цветовой палитрой текста и фона консоли, воспроизводить звуковые сигналы через встроенный динамик компьютера, а также работать с дескрипторами стандартных потоков ввода–вывода, обеспечивая гибкий контроль над отображением информации в консольном окне.

Дополнительно, для обработки ввода с клавиатуры часто используется библиотека `conio.h`, хорошо известная разработчикам Си–ориентированных приложений. Одной из наиболее востребованных функций этой библиотеки является `getch()`, позволяющая считывать нажатие клавиши без необходимости последующего подтверждения нажатием клавиши Enter. Это крайне полезно при реализации интерфейсов, в которых важна оперативная

реакция на действия пользователя – например, при перемещении по меню, выборе пунктов, управлении игровыми элементами или при реализации управления в реальном времени.

Благодаря использованию этих библиотек и их функционала становится возможной реализация целого комплекса элементов интерфейса. В частности, это касается главного меню программы, систем управления пользователями, экранов выбора режима игры, а также непосредственного интерфейса игрового процесса. Гибкие возможности управления цветом позволяют выделять активные элементы, подчеркивать текущий выбор и визуально разграничивать различные разделы экрана, тем самым делая навигацию более интуитивной. Дополнительные звуковые сигналы, генерируемые по определенным событиям, обеспечивают обратную связь и помогают пользователю не только видеть, но и слышать реакцию системы на его действия.

Таким образом, даже в текстовом формате можно создать насыщенный и функциональный интерфейс, который будет не только выполнять свои задачи, но и обеспечивать комфортное взаимодействие с приложением.

1.5 Обзор игровых алгоритмов

При создании игры в шашки на компьютере важно правильно реализовать алгоритм обработки ходов. Программа должна уметь проверять, можно ли сделать тот или иной ход, соответствует ли он правилам и не нарушает ли порядок игры.

Каждый раз, когда игрок выбирает фигуру и клетку, куда хочет походить, программа должна убедиться, что ход допустим. Например, шашка может ходить только по диагонали и только на свободные клетки. Если есть возможность побить фигуру соперника, программа должна это обнаружить и заставить игрока сделать ударный ход.

Также нужно следить за тем, чтобы шашка превращалась в дамку при достижении последней горизонтали. Дамка, в свою очередь, ходит по другим правилам, и их тоже нужно учесть.

Если игра реализует искусственный интеллект, то компьютер сам выбирает ход, проверяя все возможные варианты и выбирая лучший из них – например, тот, где он сможет побить шашку соперника.

Таким образом, игровой алгоритм помогает программе контролировать ход партии и соблюдать все правила классической игры в шашки.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Реализация алгоритма хода шашки

Функциональное проектирование логики игры в шашки опирается на формализацию игровых правил через систему взаимодействующих алгоритмов. Несмотря на внешнюю простоту одного игрового хода, его реализация требует выполнения целого ряда последовательных операций, каждая из которых обрабатывается как самостоятельная логическая единица. Системный подход предполагает разбиение общей задачи на подзадачи, решаемые с помощью отдельных функций. В рамках одного хода игрока последовательно выполняются несколько алгоритмов.

Ход для игрока и его противника реализуется по идентичному алгоритму, с корректировкой лишь отдельных условий циклов и значений, зависящих от направления движения и принадлежности фигуры. Для наглядности можно рассмотреть алгоритм выполнения хода со стороны игрока, управляющего «розовыми» шашками.

Описание логики следует начать с базовой проверки – наличия возможности сделать корректный ход с текущей позиции. Функция `It_a_free_movePLAYER` проверяет возможность совершения хода игроком, начинающим игру с нижнего края игрового поля. В рамках своей логики она анализирует наличие хотя бы одного допустимого действия: перемещения на свободную соседнюю клетку или выполнения взятия шашки соперника. Если такой ход существует, функция возвращает значение `1`, в противном случае – `0`, что означает отсутствие возможных действий на текущем этапе.

```
Int It_a_free_movePLAYER(int i,int j,int playing_field[][8],int*
blocking_counter) {
if (i > 0 && j < 7 && playing_field[i  -  1][j + 1] == 0) {
    *blocking_counter = 0;
    return 1;
}
else if (i > 0 && j > 0 && playing_field[i  -  1][j  -  1] == 0) {
    *blocking_counter = 0;    return 1;
}
else if (i > 0 && j < 7 && (playing_field[i  -  1][j + 1] == 1 ||
playing_field[i  -  1][j + 1] == 3) && (i  -  2 >= 0 && j + 2 < 6 &&
playing_field[i  -  2][j + 2] == 0)) {
    *blocking_counter = 0;
    return 1;
}
else if (i > 0 && j > 0 && (playing_field[i  -  1][j  -  1] == 1 ||
playing_field[i  -  1][j  -  1] == 3) && (i  -  1 > 0 && j  -  1 > 0 &&
playing_field[i  -  2][j  -  2] == 0)) {
    *blocking_counter = 0;
    return 1;
}
else {
    (*blocking_counter)++;
    return 0;
}
}
```

Далее необходимо реализовать сам процесс выполнения хода шашки. Рассмотрим ход влево Если ход осуществляется на пустую диагональную клетку, происходит замена значений: текущая (исходная) позиция в массиве обнуляется, а целевая клетка получает значение, соответствующее перемещенной шашке.

```
void Move_to_the_leftPLAYER(int i, int j, int playing_field[][8], char*
name_player, int* point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers, int flag) {
int mov, MOV;
Assigning_a_movePRIORITY1(priority, &mov, &MOV);

if (An_impossible_left_move_for_a_player(i, j, playing_field, priority)) {
    printf("\n\t\x1b[31m НЕВОЗМОЖНО СОВЕРШИТЬ ХОД! \x1b[0m");
    Sleep(750);
    return;

}
else if (playing_field[i  -  1][j  -  1] == 0) {
    if (playing_field[i][j] == MOV) {
        playing_field[i  -  1][j  -  1] = MOV;
    }
    else {
        playing_field[i  -  1][j  -  1] = mov;
    }
    playing_field[i][j] = 0;


    if (i  -  1 == 0 && priority == 2) {
        playing_field[i  -  1][j  -  1] = MOV;
    }
    return;
}

Assigning_a_movePRIORITY2(priority, &mov, &MOV);

if ((playing_field[i  -  1][j  -  1] == mov || playing_field[i  -  1][j  -
1] == MOV) && playing_field[i  -  2][j  -  2] == 0) {
    Taking_checkers_and_checking_moves(i, j, playing_field,
number_of_opponent_checkers, 1, name_player, point, priority,
number_of_your_checkers);
}
 }
```

В случае взятия фигуры соперника вызывается функция , которая отвечает за замену значений на игровом поле: она удаляет побитую шашку, перемещает свою на соответствующую клетку и проверяет, существует ли возможность повторного взятия. Если такая возможность имеется, функция вызывает саму себя рекурсивно, что позволяет реализовать цепочку захватов в рамках одного хода.

```
void Taking_checkers_and_checking_moves(int i, int j, int playing_field[][8],
int* number_of_opponent_checkers, int left, char* name_player, int point, int
priority, int* number_of_your_checkers) {
if (priority == 1) {
    (*number_of_your_checkers) -  - ;
}
```

```
else {
    (*number_of_opponent_checkers) -  - ;
}
Performing_a_bottom_take(&i, &j, playing_field,left, priority);
int mov, MOV;
Assigning_a_movePRIORITY2(priority, &mov, &MOV);

if (Is_an_opportunity_to_shoot_down_the_right_and_leftPLAYER(i, j,
playing_field, priority)) {
    while (1) {
        Displaying_the_game_space_on_the_screen(playing_field, name_player,
point, priority, *number_of_opponent_checkers, *number_of_your_checkers, i,
j, 1);
        int direction = _getch();
        if (direction == 75) {
            Taking_checkers_and_checking_moves(i, j, playing_field,
number_of_opponent_checkers, 1, name_player, point, priority,
number_of_your_checkers);
            break;
        }
        else if (direction == 77) {
            Taking_checkers_and_checking_moves(i, j, playing_field,
number_of_opponent_checkers, 0, name_player, point, priority,
number_of_your_checkers);

            break;
        }
    }
}
else if ((i != 0 && i != 1) && (j != 0 && j != 1) && (playing_field[i  -
1][j  -  1] == mov || playing_field[i  -  1][j  -  1] == MOV) &&
(playing_field[i  -  2][j  -  2] == 0)) {
    Taking_checkers_and_checking_moves(i, j, playing_field,
number_of_opponent_checkers, 1, name_player, point, priority,
number_of_your_checkers);
}
else if ((i != 0 && i != 1) && (j != 7 && j != 6) && (playing_field[i  -
1][j + 1] == mov || playing_field[i  -  1][j + 1] == MOV) && (playing_field[i
-  2][j + 2] == 0)) {
    Taking_checkers_and_checking_moves(i, j, playing_field,
number_of_opponent_checkers, 0, name_player, point, priority,
number_of_your_checkers);
}
}
```

Дополнительная переменная left используется для определения направления взятия – влево или вправо, что важно при обработке условий перемещения по диагонали и правильной идентификации позиции побитой фигуры.

2.2 Реализация алгоритма хода дамки

Алгоритм хода дамки во многом схож с алгоритмом обычной шашки, однако требует более расширенной проверки из – за ее увеличенных возможностей. В отличие от простой фигуры, дамка может перемещаться на любое количество клеток по диагонали, пока путь свободен, а также выполнять взятие на расстоянии. Поэтому при обработке хода необходимо учитывать возможность движения и атаки во всех четырех диагональных

направлениях.

```
int It_a_free_moveQUEEN(int i, int j, int playing_field[][8], int priority,
int* blocking_counter)
 {
if (i > 0 && j < 7 && playing_field[i  -  1][j + 1] == 0) {

*blocking_counter = 0;
 return 1;
 }
 else if (i > 0 && j > 0 && playing_field[i  -  1][j  -  1] == 0) {

*blocking_counter = 0;
 return 1; }

 else if (i < 7 && j < 7 && playing_field[i + 1][j + 1] == 0) {

*blocking_counter = 0;
 return 1;
 }
else if (i < 7 && j > 0 && playing_field[i + 1][j  -  1] == 0) {

*blocking_counter = 0;
return 1;
 }
else if (i < 7 && j < 7 && ((priority == 1 && (playing_field[i + 1][j + 1] ==
2 || playing_field[i + 1][j + 1] == 4)) || (priority == 2 && (playing_field[i
+ 1][j + 1] == 1 || playing_field[i + 1][j + 1] == 3))) && (i + 1 < 7 && j +
1 < 7 && playing_field[i + 2][j + 2] == 0)) {

*blocking_counter = 0;
 return 1;
 }
 else if (i < 7 && j >0 && (((priority == 1 || priority == 3) &&
(playing_field[i + 1][j  -  1] == 2 || playing_field[i + 1][j  -  1] == 4))
|| (priority == 2 && (playing_field[i + 1][j  -  1] == 1 || playing_field[i +
1][j  -  1] == 3))) && (i + 1 < 7 && j  -  1 > 0 && playing_field[i + 2][j  -
2] == 0)) {

 *blocking_counter = 0;
 return 1;
}
 else if (i > 0 && j < 7 && ((((priority == 1 || priority == 3) &&
(playing_field[i  -  1][j + 1] == 2 || playing_field[i  -  1][j + 1] == 4))
|| (priority == 2 && (playing_field[i  -  1][j + 1] == 1 || playing_field[i
-  1][j + 1] == 3)))) && (i  -  1 > 1 && j + 1 < 7 && playing_field[i  -
2][j + 2] == 0)) {

*blocking_counter = 0;
return 1;
}
else if (i > 0 && j > 0 && (((priority == 1 || priority == 3) &&
(playing_field[i  -  1][j  -  1] == 2 || playing_field[i  -  1][j  -  1] ==
4)) || (priority == 2 && (playing_field[i  -  1][j  -  1] == 1 ||
playing_field[i  -  1][j  -  1] == 3))) && (i  -  1 > 0 && j  -  1 > 0 &&
playing_field[i  -  2][j  -  2] == 0)) {

*blocking_counter = 0;
 return 1; }
else {

(*blocking_counter)++;
```

```
return 0; } }
```

Функция `It_a_free_moveQUEEN` возвращает значение `1`, если существует хотя бы один допустимый ход – будь то перемещение на свободную диагональ или возможность взятия фигуры соперника. Если же ни одно из направлений не дает допустимого варианта действия, функция возвращает `0`, сигнализируя об отсутствии возможных ходов для данной дамки.

Далее, в зависимости от выбора направления пользователем, осуществляется определение стороны хода дамки. После этого выполняются те же базовые действия, что и для обычной шашки.

Однако в отличие от обычной шашки, ход дамки может продолжаться до тех пор, пока пользователь не завершит его нажатием клавиши Enter, пока фигура не достигнет границы поля, либо пока не будет выполнено взятие. Эти условия определяют завершение хода, что придает управлению дамкой большую гибкость. Данные действия позволили осуществить циклы `while(1)`, а также рекурсивный вызов функций.

```
void Make_a_queens_move(int playing_field[][8], char* name_player, int*
point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers, int i, int j) {

while (1) {    Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, 2);

    int signal = _getch(), direction;

    if (signal == 72 && Checking_the_upper_direction(i, j, playing_field,
priority))
    {        while (1) {

            Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, 1);
            direction = _getch();
            if (direction == 75) {
                The_queens_move_is_down_to_the_left(playing_field,
name_player, point, priority, number_of_opponent_checkers,
number_of_your_checkers, i, j);
                break;
            }
            else if (direction == 77) {



                The_queens_move_is_down_to_the_right(playing_field,
name_player, point, priority, number_of_opponent_checkers,
number_of_your_checkers, i, j);
                break;


            }
        }        break;
    }
    else if (signal == 80 && Checking_the_lower_direction(i, j,
playing_field, priority)) {
```

```
        while (1) {

            Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, 1);

            direction = _getch();
            if (direction == 75) {

                The_queens_move_is_upp_to_the_left(playing_field,
name_player, point, priority, number_of_opponent_checkers,
number_of_your_checkers, i, j);
                break;
            }

            else if (direction == 77) {


 The_queens_move_is_upp_to_the_right(playing_field, name_player, point,
priority, number_of_opponent_checkers, number_of_your_checkers, i, j);
                break;
            }
        }
        break;

    }
}


}
```

2.3 Реализация ИИ на основе «жадного» алгоритма

Для создания элементарного искусственного интеллекта в рамках игры в шашки можно использовать один из самых простых подходов – так называемый «жадный» алгоритм. Несмотря на ограниченные интеллектуальные возможности, такие алгоритмы позволяют реализовать базовое поведение противника без сложных вычислений и глубокого анализа ситуации.

Логика работы «жадного» алгоритма заключается в выборе хода, приносящего наибольшую немедленную выгоду. Обычно это означает, что компьютер анализирует все доступные ему ходы и выбирает тот, при котором будет взята шашка противника. При этом не учитываются возможные последствия такого хода в будущем – алгоритм не просчитывает ответные действия игрока и не планирует серию ходов [3].

Условный искусственный интеллект в данном консольном приложении реализован на основе простого «жадного» алгоритма, основной целью которого является приоритетное выполнение взятия шашки соперника.

Алгоритм работает в три последовательных этапа, каждый из которых выполняет определенную задачу.

На первом этапе работы искусственного интеллекта с помощью цикла

`for` осуществляется последовательный проход по всем шашкам компьютера, размещенным на игровом поле. Для каждой фигуры выполняется проверка на возможность выполнения взятия.

Если в процессе обхода обнаруживается хотя бы одна шашка, способная побить фигуру соперника, то немедленно осуществляется данный ход – приоритетное взятие. Таким образом, выполнение первого прохода направлено исключительно на реализацию самой важной задачи жадного алгоритма: не упустить возможность нанести ущерб противнику. После успешного взятия цикл прерывается, и этап завершается.

```
int A_bot_pass_aimed_at_taking_checkers(int playing_field[][8], char*
name_player, int* point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers) {
int counter = 0;
int blocking_counter = 0;

for (int i = 0; i <= 7; i++) {
    if (blocking_counter == *number_of_opponent_checkers) {
        return 1;
    }
    for (int j = 0; j <= 7; j++) {

        if (playing_field[i][j] == 1 || playing_field[i][j] == 3) {
            counter++;

            if ((playing_field[i][j] == 1 && It_a_free_moveFRIEND(i, j,
playing_field, &blocking_counter)) || (playing_field[i][j] == 3 &&
It_a_free_moveQUEEN(i, j, playing_field, priority, &blocking_counter)))
 {


 Displaying_the_game_space_on_the_screenBOT(playing_field, name_player,
point, priority, *number_of_opponent_checkers, *number_of_your_checkers, i,
j, 1);

Beep(300, 300);
                Sleep(500);

                if (It_is_possible_to_shoot_down(i, j, playing_field,
number_of_your_checkers)) {

 Displaying_the_game_space_on_the_screenBOT(playing_field, name_player,
point, priority, *number_of_opponent_checkers, *number_of_your_checkers,
INT_MIN, INT_MIN, INT_MIN);

                    Sleep(500);
                    return 1;

                }
                else if (playing_field[i][j] == 3 &&
It_is_possible_to_shoot_downQUEEN(i, j, playing_field,
number_of_your_checkers)) {

Displaying_the_game_space_on_the_screenBOT(playing_field, name_player, point,
priority, *number_of_opponent_checkers, *number_of_your_checkers, INT_MIN,
INT_MIN, INT_MIN);
```

```
                    Sleep(500);
                    return  1;

                }

            }

        }
    }
    if (counter == *number_of_opponent_checkers) {
        return 0;
    }
}}
```

Далее, если первая функция `A_bot_pass_aimed_at_taking_checkers` не вернула значение `1`, это означает, что ни одна шашка не имеет возможности побить фигуру соперника. В этом случае управление переходит ко второму этапу – проверке возможности совершения безопасного хода.

С помощью нового цикла `for` вновь осуществляется проход по всем шашкам. Для каждой из них проверяется наличие допустимого перемещения, при котором фигура не окажется под угрозой немедленного взятия со стороны соперника на следующем ходу.

Если такой безопасный ход находится, он выполняется, и функция `The_passage_of_the_bot_aimed_at_safe_passage` возвращает значение `1`, сигнализируя об успешном завершении второго этапа. Если же ни один безопасный ход не найден, выполнение алгоритма продолжается на третьем этапе.

```
int The_passage_of_the_bot_aimed_at_safe_passage(int playing_field[][8],
char* name_player, int* point, int priority, int*
number_of_opponent_checkers, int* number_of_your_checkers) {

int counter = 0;
int blocking_counter = 0;

for (int i = 0; i <= 7; i++) {
    for (int j = 0; j <= 7; j++) {

        if (playing_field[i][j] == 1 || playing_field[i][j] == 3) {
            counter++;

            if ((playing_field[i][j] == 1 && It_a_free_moveFRIEND(i, j,
playing_field, &blocking_counter)) || (playing_field[i][j] == 3 &&
It_a_free_moveQUEEN(i, j, playing_field, priority, &blocking_counter))) {

                Displaying_the_game_space_on_the_screenBOT(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, 1);
                Sleep(500);
                Beep(300, 300);


                if (There_is_a_safe_passage(i, j, playing_field)) {

Displaying_the_game_space_on_the_screenBOT(playing_field, name_player, point,
```

```
priority, *number_of_opponent_checkers, *number_of_your_checkers, INT_MIN,
INT_MIN, INT_MIN);

                    Sleep(500);
                    return 1;

                }
            }

        }



    }

    if (counter == *number_of_opponent_checkers) {
        return 0;
    }
}

}
```

Если функция `The_passage_of_the_bot_aimed_at_safe_passage` также возвращает `0`, это означает, что ни один безопасный вариант перемещения для фигур компьютера не найден. В таком случае запускается третий и последний проход по шашкам.

На данном этапе работы программы реализуется ключевая часть алгоритма принятия решений со стороны искусственного интеллекта. С помощью очередного цикла `for` осуществляется последовательный обход всех оставшихся на поле фигур. Каждая фигура анализируется на предмет возможных допустимых ходов, соответствующих текущей позиции и правилам игры.

Как только программа обнаруживает хотя бы один допустимый ход для любой из фигур, этот ход немедленно выполняется. При этом не производится углубленный анализ последствий действия, что обусловлено стремлением сохранить активность игры и не допустить пропуска хода. Такой подход позволяет избежать ситуации, при которой искусственный интеллект «зависает» в ожидании более выгодной комбинации или теряет ход из – за отсутствия стратегического планирования.

Этот этап логики завершает выполнение алгоритма. Он заключается в том, что ИИ делает первый найденный допустимый ход, не оценивая его эффективность или потенциальные.

Схема алгоритма приведена в приложении А.

Листинг исходного кода программы представлен в приложении Б.

3 ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ

Тестирование приложения проводится на примере игры с искусственным интеллектом. Основная цель – определить работоспособность ИИ и возможности управления пользователями.

Нажимается кнопка «Играть», после чего производится удаление предыдущего игрока (см. рисунок 3.1).

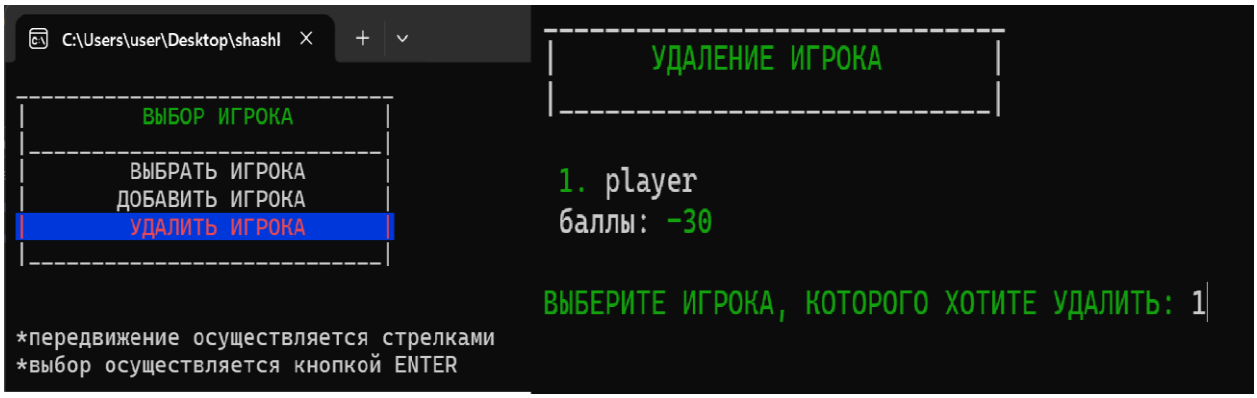

Рисунок 3.1 – Удаление игрока

Далее производится добавление нового игрока (см. рисунок 3.2).

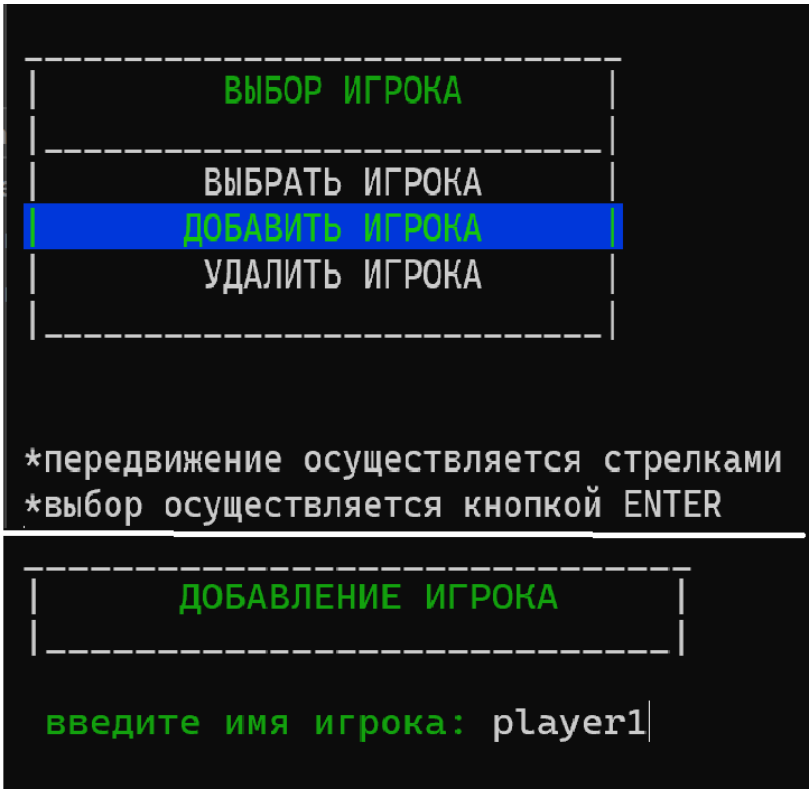

Рисунок 3.2 – Добавление игрока

Перед началом игры происходит выбор игрока (см. рисунок 3.3).

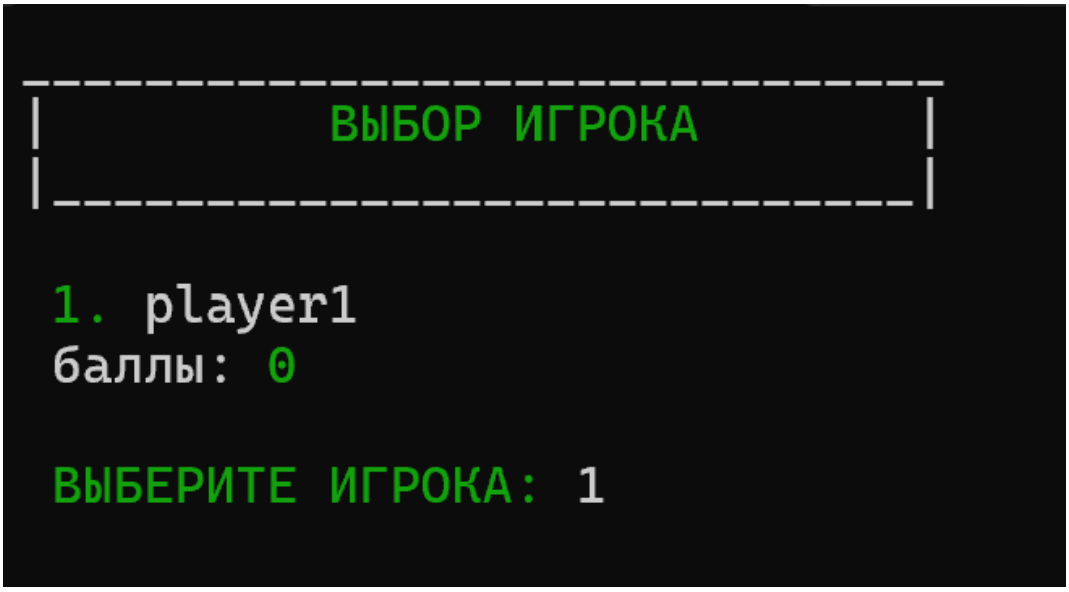

Рисунок 3.3 – Выбор игрока

Чтобы начать игру с ИИ, необходимо выбрать режим «играть с ботом». После чего начинается игровой процесс (см. рисунок 3.4).

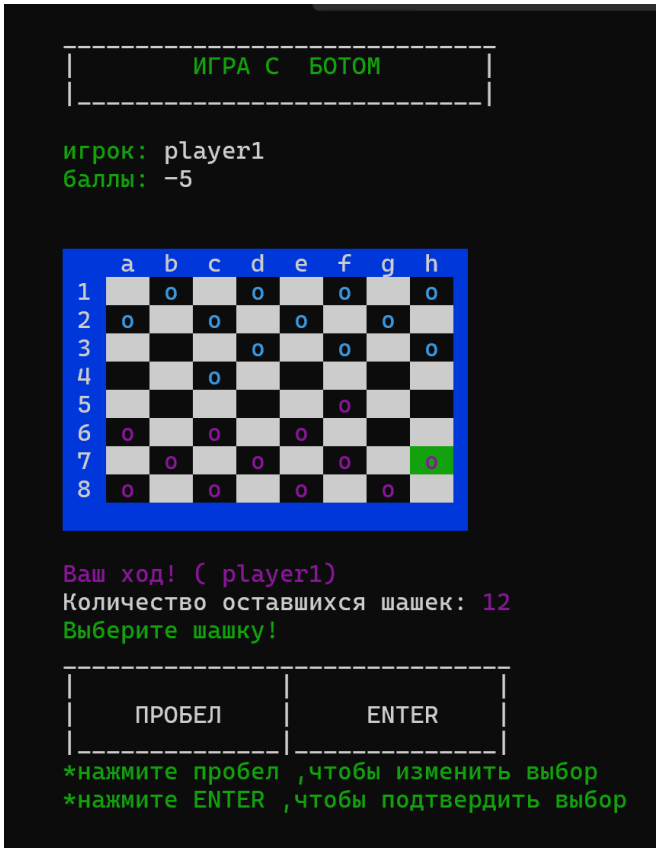

Рисунок 3.4 – Игровой процесс

По результатам игры ИИ одержал победу и у игрока отнялось 5 баллов (см. рисунок 3.5).

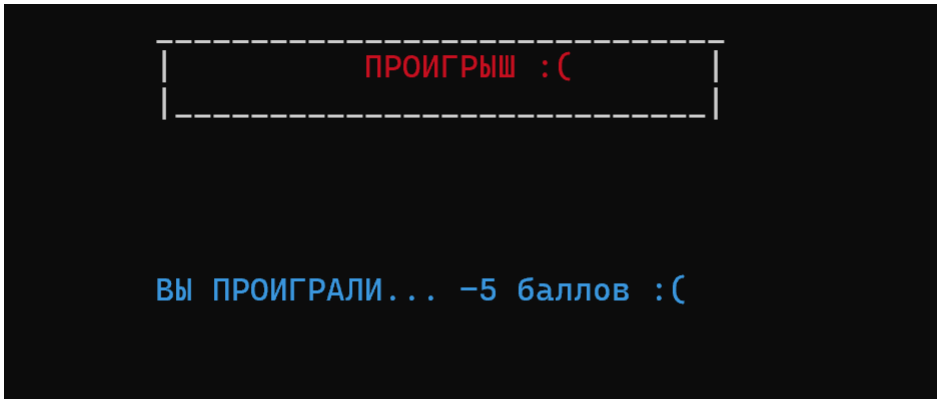

Рисунок 3.5 – Проигрыш

ЗАКЛЮЧЕНИЕ

В ходе выполнения учебной практики была успешно реализована консольная версия игры в шашки на языке программирования C. Работа над проектом позволила закрепить теоретические знания и на практике освоить важные аспекты разработки программного обеспечения: от проектирования структуры данных до создания пользовательского интерфейса и внедрения простейшего искусственного интеллекта.

Созданное приложение включает в себя все основные элементы классической игры: перемещение и взятие фигур, превращение в дамку, контроль правил и завершения партии. Особое внимание было уделено взаимодействию с пользователем, а также реализации логики принятия решений для игры с компьютером, основанной на «жадном» алгоритме.

Реализация данного проекта не только углубила навыки программирования на языке C, но и помогла лучше понять организацию игровых процессов и архитектуру консольных приложений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Григорьев, В. Н. Шашки: правила, тактика, стратегии. – Москва: АСТ, 2012. – 192 с.

[2] Правила классической игры в шашки [Электронный ресурс]. – Режим доступа: https://www.shashki.com/rules/classic/ Дата доступа: 26.03.2025.

[3] Кормен, Т. Х., Лейзерсон, Ч. Э., Ривест, Р. Л., Штайн, К. Алгоритмы: построение и анализ. – 3 – е изд. – Москва: Вильямс, 2013. – 1328 с.

[ ] Ю.А. Луцик, А.М. Ковальчук, Лукьянова И.А. – Учебное пособие по курсу «Основы алгоритмизации и программирования». – Минск: БГУИР, 2007 г.

[ ] Конспект лекций по курсу «Основы алгоритмизации и программирования» для студентов всех специальностей и всех форм обучения. – Мн.: БГУИР, 2004. – 89 с.: ил. 4.

ПРИЛОЖЕНИЕ А
(обязательное)

Схема алгоритма ИИ

ПРИЛОЖЕНИЕ Б
(обязательное)

Листинг программы

Файл Source.c

```
#include <stdio.h>
#include <locale.h>
#include"Checking.h"
#include"Main_menu.h"

int main(int argc, char** argv) {
setlocale(LC_ALL, "");
Checking_arguments(argc);
char* filepath = argv[1];
Main_menu(filepath);



return 0;

}
```

Файл Main_menu.c

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <locale.h>
#include <windows.h>
#include <string.h>
#include"Checking.h"
#include"Game_implementation.h"

void Menu_display(int position) {
system("cls");
CONSOLE_SCREEN_BUFFER_INFO csbi;
HANDLE hConsole = GetStdHandle(STD_OUTPUT_HANDLE);

if (GetConsoleScreenBufferInfo(hConsole, &csbi)) {
    WORD savedAttributes = csbi.wAttributes;

    printf("_____________________________\n");
    printf("|   \x1b[32m         ШАШКИ           \x1b[0m|\n");
    printf("|_____________________________|\n");



    if (position == 1) {
        SetConsoleTextAttribute(hConsole, FOREGROUND_GREEN |
```

```
FOREGROUND_INTENSITY | BACKGROUND_BLUE);
        printf("|            ИГРАТЬ          |\n");
        SetConsoleTextAttribute(hConsole, savedAttributes);
    }
    else {
        printf("|            ИГРАТЬ          |\n");
    }
    if (position == 2) {
        SetConsoleTextAttribute(hConsole, FOREGROUND_GREEN |
FOREGROUND_INTENSITY | BACKGROUND_BLUE);
        printf("|         ПРАВИЛА ИГРЫ       |\n");
        SetConsoleTextAttribute(hConsole, savedAttributes);
    }
    else {
        printf("|         ПРАВИЛА ИГРЫ       |\n");
    }
    if (position == 3) {
        SetConsoleTextAttribute(hConsole, FOREGROUND_RED |
FOREGROUND_INTENSITY | BACKGROUND_BLUE);
        printf("|            ВЫЙТИ           |\n");
        SetConsoleTextAttribute(hConsole, savedAttributes);
    }
    else {
        printf("|            ВЫЙТИ           |\n");
    }
    printf("|____________________________|\n");

    printf("\n\n*передвижение осуществляется стрелками \n");
    printf("*выбор осуществляется кнопкой ENTER \n");

}
else {

    printf("\n\x1b[31mошибка: НЕВОЗМОЖНО ПОЛУЧИТЬ ИНФОРМАЦИЮ О КОНСОЛИ
\x1b[0m");

}

}
void Displaying_the_game_rules_on_the_screen() { int signal; system("cls");
printf("_____________________________\n");
printf("|   \x1b[35m      ПРАВИЛА ИГРЫ \x1b[0m       |\n");
printf("|____________________________|\n");

printf("Есть два режима игры:\n");
printf(" -  На двоих\n");
printf(" -  С ботом\n");

printf("\nШаги осуществляются поочередно\n");
printf(" -  первые ходят \x1b[35mрозовые\x1b[0m \n");
printf(" -  обычной шашке нельзя ходить назад и бить назад \n");
printf(" -  если ход невозможен, очередь переходит ко второму игроку \n");
printf(" -  дамкой становится та шашка ,которая дошла до противоположного
```

```
конца поля\n");

printf("\nИгра завершается , когда шашки одного из игроков все сбиты\n");
printf(" -  если вы проиграли у вас отнимется 5 баллов \n");
printf(" -  если вы выиграли вам добавится 5 баллов \n");

while (1) {
    printf("\n\nВЫХОД  -  -  -  -  -  -  -  -  -  -  -  -  - >\x1b[32m
ENTER\x1b[0m  \n");
    signal = _getch();
    if (signal == 13) {
        break;
    }

}

}
struct PLAYER {
char* name;
int  point;

};
int Reading_and_calculating_the_size(FILE* fp, char* buffer) {
fgets(buffer, 256, fp);
int size = strlen(buffer);
buffer[size  -  1] = '\0';

return size;

}
int Memory_allocation_and_data_recording(struct PLAYER** player, FILE* fp) {
int count_players = 0;
fscanf_s(fp, "%d", &count_players, sizeof(int));
Fseek_and_checkingCUR(fp, 1);
*player = malloc(sizeof(struct PLAYER) * count_players);
Checking_memory_allocationPLAYER(player);

char buffer[256];

for (int i = 0; i < count_players; i++) {

    int size = Reading_and_calculating_the_size(fp, buffer);
    (*player)[i].name = malloc(sizeof(char) * size);
    Checking_memory_allocation((*player)[i].name);
    strcpy_s((*player)[i].name, size, buffer);

    fscanf_s(fp, "%d", &(*player)[i].point, sizeof(int));
    Fseek_and_checkingCUR(fp, 2);
}
```

```
return  count_players;

}
struct PLAYER* Opening_the_file_and_rewriting_the_players(char* filepath,
int* count_players) {
struct PLAYER* player = NULL;
FILE* fp = fopen(filepath, "r");
Checking_the_file_open(fp);

*count_players = Memory_allocation_and_data_recording(&player, fp);



Fclose_and_checking(fp);
return player;

}
void Menu_player_display(int position) {
system("cls");

CONSOLE_SCREEN_BUFFER_INFO csbi;
HANDLE hConsole = GetStdHandle(STD_OUTPUT_HANDLE);

if (GetConsoleScreenBufferInfo(hConsole, &csbi)) {
    WORD savedAttributes = csbi.wAttributes;
    printf("_______________________________\n");
    printf("|   \x1b[32m      ВЫБОР ИГРОКА       \x1b[0m|\n");
    printf("|____________________________|\n");



    if (position == 1) {
        SetConsoleTextAttribute(hConsole, FOREGROUND_GREEN |
FOREGROUND_INTENSITY | BACKGROUND_BLUE);
        printf("|        ВЫБРАТЬ ИГРОКА      |\n");
        SetConsoleTextAttribute(hConsole, savedAttributes);
    }
    else {
        printf("|        ВЫБРАТЬ ИГРОКА      |\n");
    }
    if (position == 2) {
        SetConsoleTextAttribute(hConsole, FOREGROUND_GREEN |
FOREGROUND_INTENSITY | BACKGROUND_BLUE);
        printf("|       ДОБАВИТЬ ИГРОКА      |\n");
        SetConsoleTextAttribute(hConsole, savedAttributes);
    }
    else {
        printf("|       ДОБАВИТЬ ИГРОКА      |\n");
    }
    if (position == 3) {
        SetConsoleTextAttribute(hConsole, FOREGROUND_RED |
FOREGROUND_INTENSITY | BACKGROUND_BLUE);
        printf("|        УДАЛИТЬ ИГРОКА      |\n");
        SetConsoleTextAttribute(hConsole, savedAttributes);
```

```
    }
    else {
        printf("|        УДАЛИТЬ ИГРОКА      |\n");
    }
    printf("|____________________________|\n");

    printf("\n\n*передвижение осуществляется стрелками \n");
    printf("*выбор осуществляется кнопкой ENTER \n");

}
else {

    printf("\n\x1b[31mошибка: НЕВОЗМОЖНО ПОЛУЧИТЬ ИНФОРМАЦИЮ О КОНСОЛИ
\x1b[0m");

}

}
void Displaying_the_player_on_the_screen(struct PLAYER** player, int*
count_players) {
for (int i = 0; i < *count_players; i++) {

    printf("\n \x1b[32m%d.\x1b[0m %s", i + 1, (*player)[i].name);
    printf("\n баллы: \x1b[32m%d\x1b[0m ", (*player)[i].point);

}

}
int Player_withdrawal_and_selection(struct PLAYER** player, int*
count_players) {
system("cls");

printf("_______________________________\n");
printf("|   \x1b[32m      ВЫБОР ИГРОКА       \x1b[0m|\n");
printf("|____________________________|\n");

Displaying_the_player_on_the_screen(player, count_players);

printf("\n\n \x1b[32mВЫБЕРИТЕ ИГРОКА:\x1b[0m ");
int index_player;
while (!scanf_s("%d", &index_player) || getchar() != '\n' || index_player >
*count_players || index_player < 1) {
    printf("\n \x1b[31mВВЕДИТЕ КОРРЕКТНОЕ ЗНАЧЕНИЕ: \x1b[0m ");

}
return index_player  -  1;

}
int Reading_and_calculating_the_size_STDIN(char* buffer) {
fgets(buffer, 256, stdin);
int size = strlen(buffer);
```

```
buffer[size  -  1] = '\0';

return size;

}
void Adding_a_player(struct PLAYER** player, int* count_players) {
char buffer[256];
(*count_players)++;
*player = realloc(*player, *count_players * sizeof(struct PLAYER));
Checking_memory_allocationPLAYER(player);

system("cls");

printf("_______________________________\n");
printf("|   \x1b[32m   ДОБАВЛЕНИЕ ИГРОКА     \x1b[0m|\n");
printf("|_____________________________|\n");



printf("\n\x1b[32m введите имя игрока:\x1b[0m ");
int size = Reading_and_calculating_the_size_STDIN(buffer);
(*player)[*count_players  -  1].name = malloc(sizeof(char) * size);
Checking_memory_allocation((*player)[*count_players  -  1].name);
strcpy_s((*player)[*count_players  -  1].name, size, buffer);

(*player)[*count_players  -  1].point = 0;

}
void Removing_a_player(struct PLAYER** player, int* count_players, int index)
{
free((*player)[index].name);
(*count_players) -  - ;

*player = realloc(*player, *count_players * sizeof(struct PLAYER));
if (*count_players != 0) {
    Checking_memory_allocationPLAYER(player);
}

}
void Removing_a_player_and_input_index(struct PLAYER** player, int*
count_players) {
system("cls");

printf("_______________________________\n");
printf("|   \x1b[32m   УДАЛЕНИЕ ИГРОКА       \x1b[0m|\n");
printf("|_____________________________|\n");



Displaying_the_player_on_the_screen(player, count_players);

printf("\n\n\x1b[32mВЫБЕРИТЕ ИГРОКА, КОТОРОГО ХОТИТЕ УДАЛИТЬ:\x1b[0m ");
int index_player;
```

```
while (!scanf_s("%d", &index_player) || getchar() != '\n' || index_player >
*count_players || index_player < 1) {

    printf("\n\x1b[31mВВЕДИТЕ КОРРЕКТНОЕ ЗНАЧЕНИЕ: \x1b[0m ");

}

Removing_a_player(player, count_players, index_player  -  1);

}
void Position_with_the_up_arrow(int* position) {
Beep(300, 300);
(*position) -  - ;
if (*position < 1) {
    *position = 3;
}

Menu_player_display(*position);

}
void Position_with_the_down_arrow(int* position) {
Beep(300, 300);
(*position)++;
if (*position > 3) {
    *position = 1;
}

Menu_player_display(*position);

}
int Menu_and_player_selection(struct PLAYER** player, int* count_players) {
int position = 1, signal;
Menu_player_display(position);

while (1) {
    signal = _getch();

    if (signal == 72) {

        Position_with_the_up_arrow(&position);

    }
    else if (signal == 80) {

        Position_with_the_down_arrow(&position);

    }
    else if (signal == 13) {
        if (position == 1) {
            if (*count_players == 0) {
                printf("\n\x1b[31mНЕТ ИГРОКОВ :(  Добавьте игрока!\x1b[0m");
```

```
                Sleep(1000);
            }
            else {
                return Player_withdrawal_and_selection(player,
count_players);
            }
        }
        else if (position == 2) {
            Adding_a_player(player, count_players);
        }
        else {

            if (*count_players == 0) {
                printf("\n\x1b[31mНЕТ ИГРОКОВ :(  \x1b[0m");
                Sleep(1000);
            }
            else {

                Removing_a_player_and_input_index(player, count_players);

            }

        }
        Menu_player_display(position);
    }




}

}
int Player_choice(char* filepath, struct PLAYER** player, int* count_players)
{
*player = Opening_the_file_and_rewriting_the_players(filepath,
count_players);
int player_number = Menu_and_player_selection(player, count_players);



return player_number;

}
void Writing_space(FILE* fp) {
char num = ' ';

fwrite(&num, sizeof(char), 1, fp);

}
void Saving_the_results_to_a_file(struct PLAYER* player, int count_players,
char* filepath) {
FILE* fp = fopen(filepath, "w");
Checking_the_file_open(fp);
```

```
fprintf(fp, "%d", count_players);
Writing_space(fp);

for (int i = 0; i < count_players; i++) {

    fprintf(fp, "%s\n", player[i].name);
    fprintf(fp, "%d\n", player[i].point);



}

Fclose_and_checking(fp);

}
void Free_players(struct PLAYER** player, int count_players) {
for (int i = 0; i < count_players; i++) {

    free((*player)[i].name);



}

free(*player);

}
void Game_implementation(char* filepath) {
struct PLAYER* player;
int count_players = 0;
int index_player = Player_choice(filepath, &player, &count_players);
Game(player[index_player].name, &player[index_player].point);

Saving_the_results_to_a_file(player, count_players, filepath);
Free_players(&player, count_players);

}
void Main_menu(char* filepath) {
int position = 1;
int signal;
Menu_display(position);
while (1) {
    signal = _getch();

    if (signal == 72) {
        Beep(300, 300);
        position -  - ;
        if (position < 1) {
            position = 3;
        }
        Menu_display(position);
    }
    else if (signal == 80) {
```

```
        Beep(300, 300);
        position++;
        if (position > 3) {
            position = 1;
        }
        Menu_display(position);
    }
    else if (signal == 13) {
        if (position == 3) {
            return;
        }
        else if (position == 2) {
            Displaying_the_game_rules_on_the_screen();

        }
        else {
            Game_implementation(filepath);
        }
        Menu_display(position);
    }



}

}
```

Файл Game_implementation.c

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <locale.h>
#include <windows.h>
#include <string.h>
#include"Checking.h"
#include"Graphic_elements.h"
void Menu_game_mode_selection(int position) {
system("cls");

CONSOLE_SCREEN_BUFFER_INFO csbi;
HANDLE hConsole = GetStdHandle(STD_OUTPUT_HANDLE);

if (GetConsoleScreenBufferInfo(hConsole, &csbi)) {
    WORD savedAttributes = csbi.wAttributes;
    printf("_____________________________\n");
    printf("|   \x1b[32m      ВЫБОР РЕЖИМА       \x1b[0m|\n");
    printf("|____________________________|\n");



    if (position == 1) {
        SetConsoleTextAttribute(hConsole, FOREGROUND_GREEN |
```

```
FOREGROUND_INTENSITY | BACKGROUND_BLUE);
        printf("|        ИГРА НА ДВОИХ       |\n");
        SetConsoleTextAttribute(hConsole, savedAttributes);
    }
    else {
        printf("|        ИГРА НА ДВОИХ       |\n");
    }
    if (position == 2) {
        SetConsoleTextAttribute(hConsole, FOREGROUND_GREEN |
FOREGROUND_INTENSITY | BACKGROUND_BLUE);
        printf("|        ИГРА С БОТОМ        |\n");
        SetConsoleTextAttribute(hConsole, savedAttributes);
    }
    else {
        printf("|        ИГРА С БОТОМ        |\n");
    }

    printf("|____________________________|\n");

    printf("\n\n*передвижение осуществляется стрелками \n");
    printf("*выбор осуществляется кнопкой ENTER \n");

}
else {

    printf("\n\x1b[31mошибка: НЕВОЗМОЖНО ПОЛУЧИТЬ ИНФОРМАЦИЮ О
КОНСОЛИ\x1b[0m");
}

}
int Game_mode_selection() {
int position = 1;
int signal;
Menu_game_mode_selection(position);

while (1) {
    signal = _getch();

    if (signal == 72) {
        Beep(300, 300);
        position -  - ;
        if (position < 1) {
            position = 2;
        }
        Menu_game_mode_selection(position);
    }
    else if (signal == 80) {
        Beep(300, 300);
        position++;
        if (position > 2) {
            position = 1;
        }
        Menu_game_mode_selection(position);
```

```
    }
    else if (signal == 13) {
        if (position == 1) {
            return 1;
        }
        else {
            return 2;
        }

    }
}

}
void Assigning_a_movePRIORITY1(int priority, int* mov, int* MOV) {
if (priority == 1) {
    *mov = 1;
    *MOV = 3;
}
else {
    *mov = 2;
    *MOV = 4;
}

}
void Assigning_a_movePRIORITY2(int priority, int* mov, int* MOV) {
if (priority == 2) {
    *mov = 1;
    *MOV = 3;
}
else {
    *mov = 2;
    *MOV = 4;
}

}
int It_a_free_movePLAYER(int i, int j, int playing_field[][8], int*
blocking_counter) {if (i > 0 && j < 7 && playing_field[i  -  1][j + 1] == 0)
{
    *blocking_counter = 0;
    return 1;
}
else if (i > 0 && j > 0 && playing_field[i  -  1][j  -  1] == 0) {
    *blocking_counter = 0;
    return 1;
}
else if (i > 0 && j < 7 && (playing_field[i  -  1][j + 1] == 1 ||
playing_field[i  -  1][j + 1] == 3) && (i  -  2 >= 0 && j + 2 < 6 &&
playing_field[i  -  2][j + 2] == 0)) {
    *blocking_counter = 0;
    return 1;
}
else if (i > 0 && j > 0 && (playing_field[i  -  1][j  -  1] == 1 ||
playing_field[i  -  1][j  -  1] == 3) && (i  -  1 > 0 && j  -  1 > 0 &&
playing_field[i  -  2][j  -  2] == 0)) {
```

```
    *blocking_counter = 0;
    return 1;
}
else {
    (*blocking_counter)++;
    return 0;
}

}
int Is_an_opportunity_to_shoot_down_the_right_and_leftPLAYER(int i, int j,
int playing_field[][8], int priority) { int mov, MOV;
Assigning_a_movePRIORITY2(priority, &mov, &MOV);
if ((i != 0 && i != 1) && (j != 0 && j != 1 && j != 7 && j != 6) &&
(playing_field[i  -  1][j  -  1] == mov || playing_field[i  -  1][j  -  1] ==
MOV) && playing_field[i  -  2][j  -  2] == 0 && (playing_field[i  -  1][j +
1] == mov || playing_field[i  -  1][j + 1] == MOV) && (playing_field[i  -
2][j + 2] == 0))
{
    return 1;
}
return 0;

}
void Performing_a_bottom_take(int *i,int *j,int playing_field[][8],int
left,int priority) {
int mov, MOV;
Assigning_a_movePRIORITY1(priority, &mov, &MOV);

if (left) {
    if (playing_field[*i][*j] == MOV) {
        playing_field[*i  -  2][*j  -  2] = MOV;
    }
    else {
        playing_field[*i  -  2][*j  -  2] = mov;
    }

    playing_field[*i  -  1][*j  -  1] = 0;
    playing_field[*i][*j] = 0;

    if (*i  -  2 == 0 && priority == 2) {
        playing_field[*i  -  2][*j  -  2] = 4;

    }
   ( *j)  - = 2;
}
else {
    if (playing_field[*i][*j] == MOV) {
        playing_field[*i  -  2][*j + 2] = MOV;
    }
    else {
        playing_field[*i  -  2][*j + 2] = mov;
    }
```

```
    playing_field[*i  -  1][*j + 1] = 0;
    playing_field[*i][*j] = 0;

    if (*i  -  2 == 0 && priority == 2) {
        playing_field[*i  -  2][*j + 2] = 4;

    }
    (*j)+=  2;
}
(*i) - = 2;

}
void Taking_checkers_and_checking_moves(int i, int j, int playing_field[][8],
int* number_of_opponent_checkers, int left, char* name_player, int point, int
priority, int* number_of_your_checkers) {
if (priority == 1) {
    (*number_of_your_checkers) -  - ;
}
else {
    (*number_of_opponent_checkers) -  - ;

}

Performing_a_bottom_take(&i, &j, playing_field,left, priority);
int mov, MOV;
Assigning_a_movePRIORITY2(priority, &mov, &MOV);

if (Is_an_opportunity_to_shoot_down_the_right_and_leftPLAYER(i,
j,playing_field, priority)) {
    while (1) {
        Displaying_the_game_space_on_the_screen(playing_field, name_player,
point, priority, *number_of_opponent_checkers, *number_of_your_checkers, i,
j, 1);
        int direction = _getch();
        if (direction == 75) {



            Taking_checkers_and_checking_moves(i, j, playing_field,
number_of_opponent_checkers, 1, name_player, point, priority,
number_of_your_checkers);
            break;
        }
        else if (direction == 77) {
            Taking_checkers_and_checking_moves(i, j, playing_field,
number_of_opponent_checkers, 0, name_player, point, priority,
number_of_your_checkers);

            break;
        }

    }
}
```

```
else if ((i != 0 && i != 1) && (j != 0 && j != 1) && (playing_field[i  -
1][j  -  1] == mov || playing_field[i  -  1][j  -  1] == MOV) &&
(playing_field[i  -  2][j  -  2] == 0)) {
    Taking_checkers_and_checking_moves(i, j, playing_field,
number_of_opponent_checkers, 1, name_player, point, priority,
number_of_your_checkers);
}
else if ((i != 0 && i != 1) && (j != 7 && j != 6) && (playing_field[i  -
1][j + 1] == mov || playing_field[i  -  1][j + 1] == MOV) && (playing_field[i
-  2][j + 2] == 0)) {
    Taking_checkers_and_checking_moves(i, j, playing_field,
number_of_opponent_checkers, 0, name_player, point, priority,
number_of_your_checkers);
}

}
int An_impossible_left_move_for_a_player(int i, int j, int
playing_field[][8], int priority) {
int mov, MOV;
Assigning_a_movePRIORITY1(priority, &mov, &MOV);
if (j == 0) {
    return 1;
}
else if ((playing_field[i  -  1][j  -  1] == mov || playing_field[i  -  1][j
-  1] == MOV) && (playing_field[i  -  2][j  -  2] == mov || playing_field[i
-  2][j  -  2] == MOV))
{
    return 1;
}
else if ((playing_field[i  -  1][j  -  1] == mov || playing_field[i  -  1][j
-  1] == MOV) && j  -  1 == 0) {
    return 1;

}
return 0;

}
int An_impossible_right_move_for_a_player(int i, int j, int
playing_field[][8], int priority) {
int mov, MOV;
Assigning_a_movePRIORITY1(priority, &mov, &MOV);



if (j == 7) {
    return 1;
}
else if ((playing_field[i  -  1][j + 1] == mov || playing_field[i  -  1][j +
1] == MOV) && (playing_field[i + 2][j + 2] == mov || playing_field[i + 2][j +
2] == MOV))
{
    return 1;
}
else if ((playing_field[i  -  1][j + 1] == mov || playing_field[i  -  1][j +
```

```
1] == MOV) && j + 1 == 7) {
    return 1;

}
return 0;

}
int An_impossible_left_move_for_a_friend(int i, int j, int
playing_field[][8], int priority) {
int mov, MOV;
Assigning_a_movePRIORITY1(priority, &mov, &MOV);

if (j == 0) {
    return 1;
}
else if ((playing_field[i + 1][j  -  1] == mov || playing_field[i + 1][j  -
1] == MOV) && (playing_field[i + 2][j  -  2] == mov || playing_field[i + 2][j
-  2] == MOV))
{
    return 1;
}
else if ((playing_field[i + 1][j  -  1] == mov || playing_field[i + 1][j  -
1] == MOV) && j  -  1 == 0) {
    return 1;

}
return 0;

}
int An_impossible_right_move_for_a_friend(int i, int j, int
playing_field[][8], int priority) {
int mov, MOV;
Assigning_a_movePRIORITY1(priority, &mov, &MOV);



if (j == 7) {
    return 1;
}
else if ((playing_field[i + 1][j + 1] == mov || playing_field[i + 1][j + 1]
== MOV) && (playing_field[i + 2][j + 2] == mov || playing_field[i + 2][j + 2]
== MOV))
{
    return 1;
}
else if ((playing_field[i + 1][j + 1] == mov || playing_field[i + 1][j + 1]
== MOV) && j + 1 == 7) {
    return 1;

}
return 0;

}
```

```
void Move_to_the_leftPLAYER(int i, int j, int playing_field[][8], char*
name_player, int* point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers, int flag) {
int mov, MOV;
Assigning_a_movePRIORITY1(priority, &mov,
&MOV);if(An_impossible_left_move_for_a_player(i, j, playing_field, priority))
{
    printf("\n\t\x1b[31m НЕВОЗМОЖНО СОВЕРШИТЬ ХОД! \x1b[0m");
    Sleep(750);
    return;

}
else if (playing_field[i  -  1][j  -  1] == 0) {
    if (playing_field[i][j] == MOV) {
        playing_field[i  -  1][j  -  1] = MOV;
    }
    else {
        playing_field[i  -  1][j  -  1] = mov;
    }
    playing_field[i][j] = 0;



    if (i  -  1 == 0 && priority == 2) {
        playing_field[i  -  1][j  -  1] = MOV;
    }
    return;
}

Assigning_a_movePRIORITY2(priority, &mov, &MOV);

if ((playing_field[i  -  1][j  -  1] == mov || playing_field[i  -  1][j  -
1] == MOV) && playing_field[i  -  2][j  -  2] == 0) {
    Taking_checkers_and_checking_moves(i, j, playing_field,
number_of_opponent_checkers, 1, name_player, point, priority,
number_of_your_checkers);
}

}
void Move_to_the_rightPLAYER(int i, int j, int playing_field[][8], char*
name_player, int* point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers, int flag) {
int mov, MOV;
Assigning_a_movePRIORITY1(priority, &mov, &MOV);
if (An_impossible_right_move_for_a_player(i, j, playing_field, priority)) {
    printf("\n\t\x1b[31m НЕВОЗМОЖНО СОВЕРШИТЬ ХОД! \x1b[0m");
    Sleep(750);
    return;
}
else if (playing_field[i  -  1][j + 1] == 0) {
    if (playing_field[i][j] == MOV) {
        playing_field[i  -  1][j + 1] = MOV;
    }
    else {
```

```
        playing_field[i  -  1][j + 1] = mov;
    }
    playing_field[i][j] = 0;

    if (priority == 2 && i  -  1 == 0) {
        playing_field[i  -  1][j + 1] = 4;
    }
    return;
}
Assigning_a_movePRIORITY2(priority, &mov, &MOV);

if ((playing_field[i  -  1][j + 1] == mov || playing_field[i  -  1][j + 1] ==
MOV) && playing_field[i  -  2][j + 2] == 0) {
    Taking_checkers_and_checking_moves(i, j, playing_field,
number_of_opponent_checkers, 0, name_player, point, priority,
number_of_your_checkers);
}

}
int It_a_free_moveFRIEND(int i, int j, int playing_field[][8], int*
blocking_counter) {if (i < 7 && j < 7 && playing_field[i + 1][j + 1] == 0) {
*blocking_counter = 0;
    return 1;
}
else if (i < 7 && j > 0 && playing_field[i + 1][j  -  1] == 0) {
    *blocking_counter = 0;
    return 1;
}
else if (i < 7 && j < 7 && (playing_field[i + 1][j + 1] == 2 ||
playing_field[i + 1][j + 1] == 4) && (i + 2 <= 7 && j + 2 <= 7 &&
playing_field[i + 2][j + 2] == 0)) {
    *blocking_counter = 0;
    return 1;
}
else if (i < 7 && j >0 && (playing_field[i + 1][j  -  1] == 2 ||
playing_field[i + 1][j  -  1] == 4) && (i + 1 < 7 && j  -  1 > 0 &&
playing_field[i + 2][j  -  2] == 0)) {
    *blocking_counter = 0;
    return 1;
}
else {
    (*blocking_counter)++;
    return 0;
}

}
int Is_an_opportunity_to_shoot_down_the_right_and_leftFRIEND(int i, int j,
int playing_field[][8], int priority) {
int mov, MOV;
Assigning_a_movePRIORITY2(priority, &mov, &MOV);

if ((i != 6 && i != 7) && (j != 0 && j != 1 && j != 7 && j != 6) &&
(playing_field[i + 1][j  -  1] == mov || playing_field[i + 1][j  -  1] ==
```

```
MOV) && playing_field[i + 2][j  -  2] == 0 && (playing_field[i + 1][j + 1] ==
mov || playing_field[i + 1][j + 1] == MOV) && (playing_field[i + 2][j + 2] ==
0))
{
    return 1;
}
return 0;

}
void Performing_the_upper_capture(int* i, int* j, int playing_field[][8], int
left, int priority) { int mov, MOV; Assigning_a_movePRIORITY1(priority,
&mov,&MOV);if (left) {
    if (playing_field[*i][*j] == MOV) {
        playing_field[*i + 2][*j  -  2] = MOV;
    }
    else {
        playing_field[*i + 2][*j  -  2] = mov;
    }
    playing_field[*i + 1][*j  -  1] = 0;
    playing_field[*i][*j] = 0;

    if (*i + 2 == 7 && priority == 1) {
        playing_field[*i + 2][*j  -  2] = 3;
    }
    (*j) - =2;
}
else {
    if (playing_field[*i][*j] == MOV) {
        playing_field[*i + 2][*j + 2] = MOV;
    }
    else {
        playing_field[*i + 2][*j + 2] = mov;
    }

    playing_field[*i + 1][*j + 1] = 0;
    playing_field[*i][*j] = 0;
    if (*i + 2 == 7 && priority == 1) {
        playing_field[*i + 2][*j + 2] = 3;
    }

    (*j)+= 2;
}

(*i)+= 2;

}
void Taking_checkers_and_checking_movesFRIEND(int i, int j, int
playing_field[][8], int* number_of_opponent_checkers, int left, char*
name_player, int point, int priority, int* number_of_your_checkers) {
if (priority == 2) {

    (*number_of_opponent_checkers) -  - ;
}
```

```
else {
    (*number_of_your_checkers) -  - ;

}
Performing_the_upper_capture(&i, &j, playing_field, left, priority);
int mov, MOV;
Assigning_a_movePRIORITY2(priority, &mov, &MOV);

if
(Is_an_opportunity_to_shoot_down_the_right_and_leftFRIEND(i,j,playing_field,
priority)) {
    while (1) {
        Displaying_the_game_space_on_the_screen(playing_field, name_player,
point, priority, *number_of_opponent_checkers, *number_of_your_checkers, i,
j, 1);
        int direction = _getch();
        if (direction == 75) {



            Taking_checkers_and_checking_movesFRIEND(i, j, playing_field,
number_of_opponent_checkers, 1, name_player, point, priority,
number_of_your_checkers);
            break;
        }
        else if (direction == 77) {
            Taking_checkers_and_checking_movesFRIEND(i, j, playing_field,
number_of_opponent_checkers, 0, name_player, point, priority,
number_of_your_checkers);

            break;
        }


    }
}
else if ((i != 6 && i != 7) && (j != 0 && j != 1) && (playing_field[i + 1][j
-  1] == mov || playing_field[i + 1][j  -  1] == MOV) && (playing_field[i +
2][j  -  2] == 0)) {
    Taking_checkers_and_checking_movesFRIEND(i, j, playing_field,
number_of_opponent_checkers, 1, name_player, point, priority,
number_of_your_checkers);
}
else if ((i != 6 && i != 7) && (j != 7 && j != 6) && (playing_field[i + 1][j
+ 1] == mov || playing_field[i + 1][j + 1] == MOV) && (playing_field[i + 2][j
+ 2] == 0)) {
    Taking_checkers_and_checking_movesFRIEND(i, j, playing_field,
number_of_opponent_checkers, 0, name_player, point, priority,
number_of_your_checkers);
}

}
void Move_to_the_leftFRIEND(int i, int j, int playing_field[][8], char*
name_player, int* point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers, int flag) {
```

```
int mov, MOV;
Assigning_a_movePRIORITY1(priority, &mov, &MOV);

if (An_impossible_left_move_for_a_friend(i, j, playing_field, priority)) {
printf("\n\t\x1b[31m НЕВОЗМОЖНО СОВЕРШИТЬ ХОД! \x1b[0m");
    Sleep(750);
    return;

}
else if (playing_field[i + 1][j  -  1] == 0) {
    if (playing_field[i][j] == MOV) {
        playing_field[i + 1][j  -  1] = MOV;
    }
    else {
        playing_field[i + 1][j  -  1] = mov;
    }

    playing_field[i][j] = 0;



    if (i + 1 == 7 && priority == 1) {
        playing_field[i + 1][j  -  1] = 3;
    }
    return;
}

Assigning_a_movePRIORITY2(priority, &mov, &MOV);

if ((playing_field[i + 1][j  -  1] == mov || playing_field[i + 1][j  -  1] ==
MOV) && playing_field[i + 2][j  -  2] == 0) {

    Taking_checkers_and_checking_movesFRIEND(i, j, playing_field,
number_of_opponent_checkers, 1, name_player, point, priority,
number_of_your_checkers);
}

}
void Move_to_the_rightFRIEND(int i, int j, int playing_field[][8], char*
name_player, int* point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers, int flag) {
int mov, MOV;
Assigning_a_movePRIORITY1(priority, &mov, &MOV);

if (An_impossible_right_move_for_a_friend(i, j, playing_field, priority)) {
    printf("\n\t\x1b[31m НЕВОЗМОЖНО СОВЕРШИТЬ ХОД! \x1b[0m");
    Sleep(750);
    return;

}
else if (playing_field[i + 1][j + 1] == 0) {
    if (playing_field[i][j] == MOV) {
```

```
        playing_field[i + 1][j + 1] = MOV;
    }
    else {
        playing_field[i + 1][j + 1] = mov;
    }
    playing_field[i][j] = 0;

    if (i + 1 == 7 && priority == 1) {
        playing_field[i + 1][j + 1] = 3;
    }
    return;
}

Assigning_a_movePRIORITY2(priority, &mov, &MOV);

if ((playing_field[i + 1][j + 1] == mov || playing_field[i + 1][j + 1] ==
MOV) && playing_field[i + 2][j + 2] == 0) {
    Taking_checkers_and_checking_movesFRIEND(i, j, playing_field,
number_of_opponent_checkers, 0, name_player, point, priority,
number_of_your_checkers);
}

}
int It_a_free_moveQUEEN(int i, int j, int playing_field[][8], int priority,
int* blocking_counter) { if (i > 0 && j < 7 && playing_field[i  -  1][j + 1]
== 0) {
*blocking_counter = 0;
return 1; }
 else if (i > 0 && j > 0 && playing_field[i  -  1][j  -  1] == 0) {
*blocking_counter = 0;
return 1; }
 else if (i < 7 && j < 7 && playing_field[i + 1][j + 1] == 0) {
*blocking_counter = 0;
 return 1; }
 else if (i < 7 && j > 0 && playing_field[i + 1][j  -  1] == 0) {
*blocking_counter = 0; return 1; } else if (i < 7 && j < 7 && ((priority == 1
&& (playing_field[i + 1][j + 1] == 2 || playing_field[i + 1][j + 1] == 4)) ||
(priority == 2 && (playing_field[i + 1][j + 1] == 1 || playing_field[i + 1][j
+ 1] == 3))) && (i + 1 < 7 && j + 1 < 7 && playing_field[i + 2][j + 2] == 0))
{ *blocking_counter = 0;
 return 1; }
 else if (i < 7 && j >0 && (((priority == 1 || priority == 3) &&
(playing_field[i + 1][j  -  1] == 2 || playing_field[i + 1][j  -  1] == 4))
|| (priority == 2 && (playing_field[i + 1][j  -  1] == 1 || playing_field[i +
1][j  -  1] == 3))) && (i + 1 < 7 && j  -  1 > 0 && playing_field[i + 2][j  -
2] == 0)) { *blocking_counter = 0;
 return 1; }
 else if (i > 0 && j < 7 && ((((priority == 1 || priority == 3) &&
(playing_field[i  -  1][j + 1] == 2 || playing_field[i  -  1][j + 1] == 4))
|| (priority == 2 && (playing_field[i  -  1][j + 1] == 1 || playing_field[i
-  1][j + 1] == 3)))) && (i  -  1 > 1 && j + 1 < 7 && playing_field[i  -
2][j + 2] == 0)) { *blocking_counter = 0;
return 1; }
```

```
else if (i > 0 && j > 0 && (((priority == 1 || priority == 3) &&
(playing_field[i  -  1][j  -  1] == 2 || playing_field[i  -  1][j  -  1] ==
4)) || (priority == 2 && (playing_field[i  -  1][j  -  1] == 1 ||
playing_field[i  -  1][j  -  1] == 3))) && (i  -  1 > 0 && j  -  1 > 0 &&
playing_field[i  -  2][j  -  2] == 0)) { *blocking_counter = 0;
return 1; }
else { (*blocking_counter)++; return 0;
 } }

int Checking_the_upper_direction(int i, int j, int playing_field[][8], int
priority) {
if (i > 0 && j < 7 && playing_field[i  -  1][j + 1] == 0) {
    return 1;
}
else if (i > 0 && j > 0 && playing_field[i  -  1][j  -  1] == 0) {
    return 1;
}
else if (i > 0 && j < 7 && (((priority == 1 && (playing_field[i  -  1][j + 1]
== 2 || playing_field[i  -  1][j + 1] == 4)) || (priority == 2 &&
(playing_field[i  -  1][j + 1] == 1 || playing_field[i  -  1][j + 1] == 3))))
&& (i  -  1 > 1 && j + 1 < 7 && playing_field[i  -  2][j + 2] == 0)) {


    return 1;
}
else if (i > 0 && j > 0 && ((priority == 1 && (playing_field[i  -  1][j  -
1] == 2 || playing_field[i  -  1][j  -  1] == 4)) || (priority == 2 &&
(playing_field[i  -  1][j  -  1] == 1 || playing_field[i  -  1][j  -  1] ==
3))) && (i  -  1 > 0 && j  -  1 > 0 && playing_field[i  -  2][j  -  2] == 0))
{
    return 1;
}
else {
    printf("\n\t\x1b[31m НЕВОЗМОЖНО СОВЕРШИТЬ ХОД! \x1b[0m");
    Sleep(750);
    return 0;
}

}
int Checking_the_lower_direction(int i, int j, int playing_field[][8], int
priority) { if (i < 7 && j < 7 && playing_field[i + 1][j + 1] == 0) { return
1; } else if (i < 7 && j > 0 && playing_field[i + 1][j  -  1] == 0) { return
1; } else if (i < 7 && j < 7 && ((priority == 1 && (playing_field[i + 1][j +
1] == 2 || playing_field[i + 1][j + 1] == 4)) || (priority == 2 &&
(playing_field[i + 1][j + 1] == 1 || playing_field[i + 1][j + 1] == 3))) &&
(i + 1 < 7 && j + 1 < 7 && playing_field[i + 2][j + 2] == 0)) {   return 1;
}
else if (i < 7 && j >0 && ((priority == 1 && (playing_field[i + 1][j  -  1]
== 2 || playing_field[i + 1][j  -  1] == 4)) || (priority == 2 &&
(playing_field[i + 1][j  -  1] == 1 || playing_field[i + 1][j  -  1] == 3)))
&& (i + 1 < 7 && j  -  1 > 0 && playing_field[i + 2][j  -  2] == 0)) {
    return 1;
}
else {
    printf("\n\t\x1b[31m НЕВОЗМОЖНО СОВЕРШИТЬ ХОД! \x1b[0m");
```

```
    Sleep(750);
    return 0;
}

}
int Checking_the_move_up_to_the_right(int i, int j, int playing_field[][8],
int priority) {
if (i > 0 && j < 7 && playing_field[i  -  1][j + 1] == 0) {
    return 1;
}
else if (i > 0 && j < 7 && (((priority == 1 && (playing_field[i  -  1][j + 1]
== 2 || playing_field[i  -  1][j + 1] == 4)) || (priority == 2 &&
(playing_field[i  -  1][j + 1] == 1 || playing_field[i  -  1][j + 1] == 3))))
&& (i  -  1 > 1 && j + 1 < 7 && playing_field[i  -  2][j + 2] == 0)) {

    return 1;
}
else {
    return 0;
}

}
int Checking_the_move_up_to_the_left(int i, int j, int playing_field[][8],
int priority) {
if (i > 0 && j > 0 && playing_field[i  -  1][j  -  1] == 0) {
    return 1;
}
else if (i > 0 && j > 0 && ((priority == 1 && (playing_field[i  -  1][j  -
1] == 2 || playing_field[i  -  1][j  -  1] == 4)) || (priority == 2 &&
(playing_field[i  -  1][j  -  1] == 1 || playing_field[i  -  1][j  -  1] ==
3))) && (i  -  1 > 0 && j  -  1 > 0 && playing_field[i  -  2][j  -  2] == 0))
{
    return 1;
}
else {
    return 0;
}

}
int Checking_the_move_up_to_the_right(int i, int j, int playing_field[][8],
int priority) {
if (i < 7 && j < 7 && playing_field[i + 1][j + 1] == 0) {
    return 1;
}
else if (i < 7 && j < 7 && ((priority == 1 && (playing_field[i + 1][j + 1] ==
2 || playing_field[i + 1][j + 1] == 4)) || (priority == 2 && (playing_field[i
+ 1][j + 1] == 1 || playing_field[i + 1][j + 1] == 3))) && (i + 1 < 7 && j +
1 < 7 && playing_field[i + 2][j + 2] == 0)) {

    return 1;
}
else {
    return 0;
```

```
}

}
int Checking_the_move_down_to_the_right(int i, int j, int playing_field[][8],
int priority) {
if (i < 7 && j < 7 && playing_field[i + 1][j + 1] == 0) {
    return 1;
}
else if (i < 7 && j < 7 && ((priority == 1 && (playing_field[i + 1][j + 1] ==
2 || playing_field[i + 1][j + 1] == 4)) || (priority == 2 && (playing_field[i
+ 1][j + 1] == 1 || playing_field[i + 1][j + 1] == 3))) && (i + 1 < 7 && j +
1 < 7 && playing_field[i + 2][j + 2] == 0)) {

    return 1;
}
else {
    return 0;
}

}
int Checking_the_move_down_to_the_left(int i, int j, int playing_field[][8],
int priority) {
if (i < 7 && j > 0 && playing_field[i + 1][j  -  1] == 0) {
    return 1;
}
else if (i < 7 && j >0 && ((priority == 1 && (playing_field[i + 1][j  -  1]
== 2 || playing_field[i + 1][j  -  1] == 4)) || (priority == 2 &&
(playing_field[i + 1][j  -  1] == 1 || playing_field[i + 1][j  -  1] == 3)))
&& (i + 1 < 7 && j  -  1 > 0 && playing_field[i + 2][j  -  2] == 0)) {
    return 1;
}
else {
    return 0;
}

}
void The_queens_move_is_down_to_the_left(int playing_field[][8], char*
name_player, int* point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers, int i, int j) {
Move_to_the_leftPLAYER(i, j, playing_field, name_player, point, priority,
number_of_opponent_checkers, number_of_your_checkers, 1);
i = i  -  1;
j = j  -  1;
while (1) {
    Displaying_the_game_space_on_the_screen(playing_field, name_player,
point, priority, *number_of_opponent_checkers, *number_of_your_checkers, i,
j, 1);
    if (Checking_the_move_up_to_the_left(i, j, playing_field, priority)) {
        while (1) {
            int move = _getch();
            if (move == 13) {
                Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, INT_MIN);
```

```
                Sleep(500);
                break;

            }
            else if (move == 75) {
                int buf1 = *number_of_opponent_checkers, buf2 =
*number_of_your_checkers;
                Move_to_the_leftPLAYER(i, j, playing_field, name_player,
point, priority, number_of_opponent_checkers, number_of_your_checkers, 1);
                i = i  -  1;
                j = j  -  1;
                if ((*number_of_opponent_checkers == buf1 ||
*number_of_your_checkers == buf2) && Checking_the_move_up_to_the_left(i, j,
playing_field, priority)) {

                    Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, 1);
                    continue;
                }
                else {
                    Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, INT_MIN);
                    Sleep(500);
                    break;
                }
            }

        }
        break;
    }
    else {
        break;
    }
}

}
void The_queens_move_is_down_to_the_right(int playing_field[][8], char*
name_player, int* point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers, int i, int j) {while (1) {

    Move_to_the_rightPLAYER(i, j, playing_field, name_player, point,
priority, number_of_opponent_checkers, number_of_your_checkers, 1);
    i = i  -  1;
    j = j + 1;
    Displaying_the_game_space_on_the_screen(playing_field, name_player,
point, priority, *number_of_opponent_checkers, *number_of_your_checkers, i,
j, 1);
    if (Checking_the_move_up_to_the_right(i, j, playing_field, priority)) {
        while (1) {
            int move = _getch();
            if (move == 13) {
```

```
                Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, INT_MIN);
                Sleep(500);
                break;


            }
            else if (move == 77) {
                int buf1 = *number_of_opponent_checkers, buf2 =
*number_of_your_checkers;
                Move_to_the_rightPLAYER(i, j, playing_field, name_player,
point, priority, number_of_opponent_checkers, number_of_your_checkers, 1);
                i = i  -  1;
                j = j + 1;
                if ((*number_of_opponent_checkers == buf1 ||
*number_of_your_checkers == buf2) && Checking_the_move_up_to_the_right(i, j,
playing_field, priority)) {

                    Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, 1);
                    continue;


                }
                else {
                    Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, INT_MIN);
                    Sleep(500);
                    break;
                }
            }
        }
        break;
    }
    else {
        break;
    }
}
 }

void The_queens_move_is_upp_to_the_left(int playing_field[][8], char*
name_player, int* point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers, int i, int j) {
while (1) {
    Move_to_the_leftFRIEND(i, j, playing_field, name_player, point, priority,
number_of_opponent_checkers, number_of_your_checkers, 1);
    i = i + 1;
    j = j  -  1;
    Displaying_the_game_space_on_the_screen(playing_field, name_player,
point, priority, *number_of_opponent_checkers, *number_of_your_checkers, i,
j, 1);
    if (Checking_the_move_down_to_the_left(i, j, playing_field, priority)) {
```

```
        while (1) {
            int move = _getch();
            if (move == 13) {
                Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, INT_MIN);
                Sleep(500);
                break;


            }
            else if (move == 75) {
                int buf1 = *number_of_opponent_checkers, buf2 =
*number_of_your_checkers;
                Move_to_the_leftFRIEND(i, j, playing_field, name_player,
point, priority, number_of_opponent_checkers, number_of_your_checkers, 1);
                i = i + 1;
                j = j  -  1;
                if ((*number_of_opponent_checkers == buf1 ||
*number_of_your_checkers == buf2) && Checking_the_move_down_to_the_left(i, j,
playing_field, priority)) {
                    Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, 1);
                    continue;


                }
                else {
                    Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, INT_MIN);
                    Sleep(500);
                    break;
                }
            }
        }
        break;
    }
    else {
        break;
    }
}

}
void The_queens_move_is_upp_to_the_right(int playing_field[][8], char*
name_player, int* point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers, int i, int j) {while (1) {
    Move_to_the_rightFRIEND(i, j, playing_field, name_player, point,
priority, number_of_opponent_checkers, number_of_your_checkers, 1);
    i = i + 1;
    j = j + 1;
    Displaying_the_game_space_on_the_screen(playing_field, name_player,
point, priority, *number_of_opponent_checkers, *number_of_your_checkers, i,
j, 1);
    if (Checking_the_move_down_to_the_right(i, j, playing_field, priority)) {
```

```
        while (1) {
            int move = _getch();
            if (move == 13) {
                Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
number_of_your_checkers, i, j, INT_MIN);
                Sleep(500);
                break;


            }
            else if (move == 77) {
                int buf1 = *number_of_opponent_checkers, buf2 =
*number_of_your_checkers;
                Move_to_the_rightFRIEND(i, j, playing_field, name_player,
point, priority, number_of_opponent_checkers, number_of_your_checkers, 1);
                i = i + 1;
                j = j + 1;
                if ((*number_of_opponent_checkers == buf1 ||
*number_of_your_checkers == buf2) && Checking_the_move_down_to_the_right(i,
j, playing_field, priority)) {
                    Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, 1);
                    continue;
                }
                else {
                    Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, INT_MIN);
                    Sleep(500);
                    break;
                }
            }
        }
        break;
    }
    else {
        break;
    }
}

}
void Make_a_queens_move(int playing_field[][8], char* name_player, int*
point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers, int i, int j) {
ile (1) {    Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, 2);
    int signal = _getch(), direction;

    if (signal == 72 && Checking_the_upper_direction(i, j, playing_field,
priority))
    {
        while (1) {
```

```
            Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, 1);
            direction = _getch();
            if (direction == 75) {
                The_queens_move_is_down_to_the_left(playing_field,
name_player, point, priority, number_of_opponent_checkers,
number_of_your_checkers, i, j);
                break;
            }
            else if (direction == 77) {



                The_queens_move_is_down_to_the_right(playing_field,
name_player, point, priority, number_of_opponent_checkers,
number_of_your_checkers, i, j);
                break;

            }
        }
        break;
    }
    else if (signal == 80 && Checking_the_lower_direction(i, j,
playing_field, priority)) {

        while (1) {

            Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, 1);
            direction = _getch();
            if (direction == 75) {

                The_queens_move_is_upp_to_the_left(playing_field,
name_player, point, priority, number_of_opponent_checkers,
number_of_your_checkers, i, j);
                break;
            }
            else if (direction == 77) {

                The_queens_move_is_upp_to_the_right(playing_field,
name_player, point, priority, number_of_opponent_checkers,
number_of_your_checkers, i, j);
                break;




            }
        }
        break;
```

```
    }

}

}
void Players_turn(int playing_field[][8], char* name_player, int* point, int
priority, int* number_of_opponent_checkers, int* number_of_your_checkers) {
int blocking_counter = 0;
int flag = 0;
int counter = 0;

for (int i = 7; i >= 0; i -  - ) {
    if (blocking_counter == *number_of_your_checkers) {        break;
    }    for (int j = 7; j >= 0; j -  - ) {
        if (playing_field[i][j] == 2 || playing_field[i][j] == 4) {
            counter++;

            if ((playing_field[i][j] == 2 && It_a_free_movePLAYER(i, j,
playing_field, &blocking_counter)) || (playing_field[i][j] == 4 &&
It_a_free_moveQUEEN(i, j, playing_field, priority, &blocking_counter))) {
                while (1) {

                    Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, flag);
                    int signal = _getch();
                    if (signal == 32) {
                        Beep(400, 300);
                        break;
                    }
                    else if (signal == 13) {
                        Beep(500, 300);
                        flag = 1;
                        if (playing_field[i][j] == 4) {

Displaying_the_game_space_on_the_screen(playing_field, name_player, point,
priority, *number_of_opponent_checkers, *number_of_your_checkers, i, j, 2);
                            Make_a_queens_move(playing_field, name_player,
point, priority, number_of_opponent_checkers, number_of_your_checkers, i, j);
                        }
                        else {
                            while (1) {

Displaying_the_game_space_on_the_screen(playing_field, name_player, point,
priority, *number_of_opponent_checkers, *number_of_your_checkers, i, j,
flag);
                                int direction = _getch();
                                if (direction == 75) {
                                    Move_to_the_leftPLAYER(i, j,
playing_field, name_player, point, priority, number_of_opponent_checkers,
number_of_your_checkers, flag);
```

```
Displaying_the_game_space_on_the_screen(playing_field, name_player, point,
priority, *number_of_opponent_checkers, *number_of_your_checkers, INT_MIN,
INT_MIN, INT_MIN);
                                    Sleep(500);
                                    break;
                                }
                                else if (direction == 77) {
                                    Move_to_the_rightPLAYER(i, j,
playing_field, name_player, point, priority, number_of_opponent_checkers,
number_of_your_checkers, flag);

Displaying_the_game_space_on_the_screen(playing_field, name_player, point,
priority, *number_of_opponent_checkers, *number_of_your_checkers, INT_MIN,
INT_MIN, INT_MIN);
                                    Sleep(500);
                                    break;
                                }
                            }
                        }
                        break;
                    }
                }

            }

        }

        if (counter == *number_of_your_checkers) {
            i = 8;
            counter = 0;
        }
        if (flag) {
            break;
        }

    }
    if (flag) {
        break;
    }
}


}
void Friends_turn(int playing_field[][8], char* name_player, int* point, int
priority, int* number_of_opponent_checkers, int* number_of_your_checkers) {
int flag = 0;
int counter = 0;
int blocking_counter = 0;
for (int i = 0; i <= 7; i++) {    if (blocking_counter ==
*number_of_opponent_checkers) {        break;    }    for (int j = 0; j <= 7;
j++) {
```

```
        if (playing_field[i][j] == 1 || playing_field[i][j] == 3) {
            counter++;

            if ((playing_field[i][j] == 1 && It_a_free_moveFRIEND(i, j,
playing_field, &blocking_counter)) || (playing_field[i][j] == 3 &&
It_a_free_moveQUEEN(i, j, playing_field, priority, &blocking_counter))) {
                while (1) {

                    Displaying_the_game_space_on_the_screen(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, flag);
                    int signal = _getch();
                    if (signal == 32) {
                        Beep(400, 300);
                        break;
                    }
                    else if (signal == 13) {
                        Beep(500, 300);
                        flag = 1;
                        if (playing_field[i][j] == 3) {

Displaying_the_game_space_on_the_screen(playing_field, name_player, point,
priority, *number_of_opponent_checkers, *number_of_your_checkers, i, j, 2);
                            Make_a_queens_move(playing_field, name_player,
point, priority, number_of_opponent_checkers, number_of_your_checkers, i, j);

                        }
                        else {
                            while (1) {

Displaying_the_game_space_on_the_screen(playing_field, name_player, point,
priority, *number_of_opponent_checkers, *number_of_your_checkers, i, j,
flag);
                                int direction = _getch();
                                if (direction == 75) {
                                    Move_to_the_leftFRIEND(i, j,
playing_field, name_player, point, priority, number_of_opponent_checkers,
number_of_your_checkers, flag);

Displaying_the_game_space_on_the_screen(playing_field, name_player, point,
priority, *number_of_opponent_checkers, *number_of_your_checkers, INT_MIN,
INT_MIN, INT_MIN);
                                    Sleep(500);
                                    break;
                                }
                                else if (direction == 77) {
                                    Move_to_the_rightFRIEND(i, j,
playing_field, name_player, point, priority, number_of_opponent_checkers,
number_of_your_checkers, flag);

Displaying_the_game_space_on_the_screen(playing_field, name_player, point,
priority, *number_of_opponent_checkers, *number_of_your_checkers, INT_MIN,
INT_MIN, INT_MIN);
```

```
                                    Sleep(500);
                                    break;
                                }

                            }
                        }
                        break;
                    }

}

            }

        }       if (counter == *number_of_opponent_checkers) {
            i =  - 1;
            counter = 0;
        }
        if (flag) {
            break;
        }

    }
    if (flag) {
        break;
    }
}void A_game_for_two(int playing_field[][8], char* name_player, int* point)
{int number_of_opponent_checkers = 12;
int number_of_your_checkers = 12;

int priority = 2;



Displaying_the_game_space_on_the_screen(playing_field, name_player, point,
priority, number_of_opponent_checkers, number_of_your_checkers, INT_MIN,
INT_MIN, INT_MIN);




while (number_of_opponent_checkers != 0 && number_of_your_checkers != 0){
    if (priority == 2) {
        Players_turn(playing_field, name_player, point, priority,
&number_of_opponent_checkers, &number_of_your_checkers);

        priority = 1;

    }
    else if (priority == 1) {
        Friends_turn(playing_field, name_player, point, priority,
&number_of_opponent_checkers, &number_of_your_checkers);
```

```
        priority = 2;
    }



}

if (number_of_opponent_checkers == 0) {

    (*point) += 5;
    Victory_screen();
}
else {
    (*point)  - = 5;
    Loss_screen();
}

}
void Initializing_the_downward_stroke(int i, int j, int*
number_of_your_checkers, int playing_field[][8], int flag) {
if (i + 2 == 7) {
   if (flag) {
        playing_field[i + 2][j + 2] = 3;
    }
    else {
        playing_field[i + 2][j  -  2] = 3;
    }
}
else if (playing_field[i][j] == 3) {
    if (flag) {
        playing_field[i + 2][j + 2] = 3;
    }
    else {
        playing_field[i + 2][j  -  2] = 3;
    }
}
else {
    if (flag) {
        playing_field[i + 2][j + 2] = 1;
    }
    else {
        playing_field[i + 2][j  -  2] = 1;
    }
}
playing_field[i][j] = 0;
(*number_of_your_checkers) -  - ;

}
int It_is_possible_to_shoot_down(int i, int j, int playing_field[][8], int*
number_of_your_checkers) {
if (i < 7 && j < 7 && (playing_field[i + 1][j + 1] == 2 || playing_field[i +
1][j + 1] == 4) && playing_field[i + 2][j + 2] == 0) {
    playing_field[i + 1][j + 1] = 0;    Initializing_the_downward_stroke(i,
j, number_of_your_checkers, playing_field, 1);
```

```
    i += 2;
    j += 2;
    It_is_possible_to_shoot_down(i, j, playing_field,
number_of_your_checkers);
    return 1;

}
else if (i < 7 && j >1 && (playing_field[i + 1][j  -  1] == 2 ||
playing_field[i + 1][j  -  1] == 4) && playing_field[i + 2][j  -  2] == 0) {

    playing_field[i + 1][j  -  1] = 0;

    Initializing_the_downward_stroke(i, j, number_of_your_checkers,
playing_field, 0);
    i += 2;
    j  - = 2;
    It_is_possible_to_shoot_down(i, j, playing_field,
number_of_your_checkers);
    return 1;

}
else if (playing_field[i][j] == 3 && i > 1 && j > 1 && (playing_field[i  -
1][j  -  1] == 2 || playing_field[i  -  1][j  -  1] == 4) && playing_field[i
-  2][j  -  2] == 0) {

    playing_field[i  -  1][j  -  1] = 0;
    playing_field[i][j] = 0;
    playing_field[i  -  2][j  -  2] = 3;
    (*number_of_your_checkers) -  - ;
    i  - = 2;
    j  - = 2;
    It_is_possible_to_shoot_down(i, j, playing_field,
number_of_your_checkers);
    return 1;
}
else if (playing_field[i][j] == 3 && i > 1 && j < 6 && (playing_field[i  -
1][j + 1] == 2 || playing_field[i  -  1][j + 1] == 4) && playing_field[i  -
2][j + 2] == 0) {

    playing_field[i  -  1][j + 1] = 0;
    playing_field[i][j] = 0;
    playing_field[i  -  2][j + 2] = 3;
    (*number_of_your_checkers) -  - ;
    i  - = 2;
    j += 2;
    It_is_possible_to_shoot_down(i, j, playing_field,
number_of_your_checkers);
    return 1;



}
return 0;
```

```
}
void Initializing_the_downward_strokeQUEEN(int i, int j, int x, int y, int*
number_of_your_checkers, int playing_field[][8], int flag) {
if (flag) {    playing_field[i + 1][j + 1] = 0;
    playing_field[i + 2][j + 2] = 3;



}
else if (flag == 2) {
    playing_field[i  -  1][j  -  1] = 0;
    playing_field[i  -  2][j  -  2] = 3;

}
else if (flag == 3) {
    playing_field[i  -  1][j + 1] = 0;
    playing_field[i  -  2][j + 2] = 3;
}
else {
    playing_field[i + 1][j  -  1] = 0;
    playing_field[i + 2][j  -  2] = 3;
}

playing_field[i][j] = 0;
playing_field[x][y] = 0;
(*number_of_your_checkers) -  - ;

}
int It_is_possible_to_shoot_downQUEEN(int x, int y, int playing_field[][8],
int* number_of_your_checkers) {
if (x < 7 && y < 7 && playing_field[x + 1][y + 1] == 0) {
    int i = x, j = y;    for (i, j; j < 6 && i < 6; i++, j++) {

        if ((playing_field[i + 1][j + 1] == 2 || playing_field[i + 1][j + 1]
== 4) && playing_field[i + 2][j + 2] == 0) {
            Initializing_the_downward_strokeQUEEN(i, j, x, y,
number_of_your_checkers, playing_field, 1);

            i += 2;
            j += 2;

            It_is_possible_to_shoot_down(i, j, playing_field,
number_of_your_checkers);
            return 1;

        }
        else if ((playing_field[i + 1][j + 1] == 2 || playing_field[i + 1][j
+ 1] == 4) && (playing_field[i + 2][j + 2] == 2 || playing_field[i + 2][j +
2] == 4)) {

            return 0;
        }
```

```
        else {
            continue;
        }
 }
}
else if (x < 7 && y >1 && playing_field[x + 1][y  -  1] == 0) {

    int i = x, j = y;
    for (i, j; i < 6 && j>1; i += 2, j  - = 2) {

        if ((playing_field[i + 1][j  -  1] == 2 || playing_field[i + 1][j  -
1] == 4) && playing_field[i + 2][j  -  2] == 0) {
            Initializing_the_downward_strokeQUEEN(i, j, x, y,
number_of_your_checkers, playing_field, 0);
            i += 2;
            j  - = 2;
            It_is_possible_to_shoot_down(i, j, playing_field,
number_of_your_checkers);
            return 1;

        }
        else if ((playing_field[i + 1][j  -  1] == 2 || playing_field[i +
1][j  -  1] == 4) && (playing_field[i + 2][j  -  2] == 2 || playing_field[i +
2][j  -  2] == 4)) {
            return 0;
        }
        else {
            continue;
        }

    }

}
else if (x > 1 && y > 1 && playing_field[x  -  1][y  -  1] == 0) {

    int i = x, j = y;
    for (i, j; i > 1 && j > 1; i  - = 2, j  - = 2) {

        if ((playing_field[i  -  1][j  -  1] == 2 || playing_field[i  -  1][j
-  1] == 4) && playing_field[i  -  2][j  -  2] == 0) {
            Initializing_the_downward_strokeQUEEN(i, j, x, y,
number_of_your_checkers, playing_field, 2);
            i  - = 2;
            j  - = 2;

            It_is_possible_to_shoot_down(i, j, playing_field,
number_of_your_checkers);

            return 1;
        }
        else if ((playing_field[i  -  1][j  -  1] == 2 || playing_field[i  -
1][j  -  1] == 4) && (playing_field[i  -  2][j  -  2] == 2 || playing_field[i
```

```
-  2][j  -  2] == 4)) {
            return 0;
        }
        else {
            continue;
        }

    }



}
else if (x > 1 && y < 1 && playing_field[x  -  1][y + 1] == 0) {

    int i = x, j = y;
    for (i, j; i > 1 && j < 6; i  - = 2, j += 2) {

        if (i > 1 && j < 1 && (playing_field[i  -  1][j + 1] == 2 ||
playing_field[i  -  1][j + 1] == 4) && playing_field[i  -  2][j + 2] == 0) {
            Initializing_the_downward_strokeQUEEN(i, j, x, y,
number_of_your_checkers, playing_field, 3);
            i  - = 2;
            j += 2;
            It_is_possible_to_shoot_down(i, j, playing_field,
number_of_your_checkers);
            return 1;
        }

        else if ((playing_field[i  -  1][j + 1] == 2 || playing_field[i  -
1][j + 1] == 4) && (playing_field[i  -  2][j + 2] == 2 || playing_field[i  -
2][j + 2] == 4)) {
            return 0;
        }
        else {
            continue;
        }

    }

}
return 0;

}
int There_is_a_safe_passage(int i, int j, int playing_field[][8]) {
if (((i < 7 && j < 7 && j != 6 && (playing_field[i + 2][j + 2] == 0 ||
(playing_field[i + 2][j + 2] == 1 || playing_field[i + 2][j + 2] == 3)) || (j
== 6 && i < 7))) && playing_field[i + 1][j + 1] == 0) {    if
(playing_field[i][j] == 3) {        playing_field[i + 1][j + 1] = 3;
    }    else {
        playing_field[i + 1][j + 1] = 1;
    }
    playing_field[i][j] = 0;
    return 1;
```

```
}
else if (((i < 7 && j >0 && j != 1 && (playing_field[i + 2][j  -  2] == 0 ||
(playing_field[i + 2][j  -  2] == 1 || playing_field[i + 2][j  -  2] == 3)))
|| (j == 1 && j > 0)) && playing_field[i + 1][j  -  1] == 0) {

    if (playing_field[i][j] == 3) {
        playing_field[i + 1][j  -  1] = 3;
    }
    else {
        playing_field[i + 1][j  -  1] = 1;
    }
    playing_field[i][j] = 0;
    return 1;
}
else if (((i > 0 && j > 0 && j != 1 && (playing_field[i  -  2][j  -  2] == 0
|| (playing_field[i  -  2][j  -  2] == 1 || playing_field[i  -  2][j  -  2]
== 3))) || (j == 1 && i > 0)) && playing_field[i][j] == 3 && playing_field[i
-  1][j  -  1] == 0) {



    playing_field[i  -  1][j  -  1] = 3;
    playing_field[i][j] = 0;
    return 1;
}
else if (((i > 0 && j < 7 && j != 6 && (playing_field[i  -  2][j + 2] == 0 ||
(playing_field[i  -  2][j + 2] == 1 || playing_field[i  -  2][j + 2] == 3)))
|| (i > 0 && j == 6)) && playing_field[i][j] == 3 && playing_field[i  -  1][j
+ 1] == 0)
{
    playing_field[i  -  1][j + 1] = 3;
    playing_field[i][j] = 0;
    return 1;
}
return 0;

}
void There_is_a_dangerous_move(int i, int j, int playing_field[][8]) {if (i <
7 && j < 7 && playing_field[i + 1][j + 1] == 0) {
    if (playing_field[i][j] == 3) {        playing_field[i + 1][j + 1] = 3;
    }
    else {
        playing_field[i + 1][j + 1] = 1;
    }
    playing_field[i][j] = 0;

}
else if (i < 7 && j >0 && playing_field[i + 1][j  -  1] == 0) {

    if (playing_field[i][j] == 3) {
        playing_field[i + 1][j  -  1] = 3;
    }
    else {
        playing_field[i + 1][j  -  1] = 1;
```

```
    }
    playing_field[i][j] = 0;

}
else if (i > 0 && j > 0 && playing_field[i  -  1][j  -  1] == 0 &&
playing_field[i][j] == 3) {



    playing_field[i  -  1][j  -  1] = 3;
    playing_field[i][j] = 0;

}
else if (i > 0 && j < 7 && playing_field[i  -  1][j + 1] == 0 &&
playing_field[i][j] == 3)
{
    playing_field[i  -  1][j + 1] = 3;
    playing_field[i][j] = 0;

}

}
void Players_turn_with_a_BOT(int playing_field[][8], char* name_player, int*
point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers) {
int blocking_counter = 0;
int flag = 0;
int counter = 0;
for (int i = 7; i >= 0; i -  - ) {
    if (blocking_counter == *number_of_your_checkers) {
        break;
    }    for (int j = 7; j >= 0; j -  - ) {

        if (playing_field[i][j] == 2 || playing_field[i][j] == 4) {
            counter++;

            if ((playing_field[i][j] == 2 && It_a_free_movePLAYER(i, j,
playing_field, &blocking_counter)) || (playing_field[i][j] == 4 &&
It_a_free_moveQUEEN(i, j, playing_field, priority, &blocking_counter))) {
                while (1) {

                    Displaying_the_game_space_on_the_screenBOT(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, flag);
                    int signal = _getch();
                    if (signal == 32) {
                        Beep(400, 300);
                        break;
                    }
                    else if (signal == 13) {
                        Beep(500, 300);
                        flag = 1;
                        if (playing_field[i][j] == 4) {
```

```
Displaying_the_game_space_on_the_screenBOT(playing_field, name_player, point,
priority, *number_of_opponent_checkers, *number_of_your_checkers, i, j, 2);
                            Make_a_queens_move(playing_field, name_player,
point, priority, number_of_opponent_checkers, number_of_your_checkers, i, j);
                        }
                        else {
                            while (1) {

Displaying_the_game_space_on_the_screenBOT(playing_field, name_player, point,
priority, *number_of_opponent_checkers, *number_of_your_checkers, i, j,
flag);
                                int direction = _getch();
                                if (direction == 75) {
                                    Move_to_the_leftPLAYER(i, j,
playing_field, name_player, point, priority, number_of_opponent_checkers,
number_of_your_checkers, flag);

Displaying_the_game_space_on_the_screenBOT(playing_field, name_player, point,
priority, *number_of_opponent_checkers, *number_of_your_checkers, INT_MIN,
INT_MIN, INT_MIN);
                                    Sleep(500);
                                    break;
                                }
                                else if (direction == 77) {
                                    Move_to_the_rightPLAYER(i, j,
playing_field, name_player, point, priority, number_of_opponent_checkers,
number_of_your_checkers, flag);

Displaying_the_game_space_on_the_screenBOT(playing_field, name_player, point,
priority, *number_of_opponent_checkers, *number_of_your_checkers, INT_MIN,
INT_MIN, INT_MIN);
                                    Sleep(500);
                                    break;
                                }
                            }
                        }
                        break;
                    }
                }

            }

        }

        if (counter == *number_of_your_checkers) {
            i = 8;
            counter = 0;
        }
        if (flag) {
            break;
        }
```

```
    }
    if (flag) {
        break;
    }
}

}
int A_bot_pass_aimed_at_taking_checkers(int playing_field[][8], char*
name_player, int* point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers) {
int counter = 0;
int blocking_counter = 0;

for (int i = 0; i <= 7; i++) {    if (blocking_counter ==
*number_of_opponent_checkers) {
        return 1;
    }
    for (int j = 0; j <= 7; j++) {

        if (playing_field[i][j] == 1 || playing_field[i][j] == 3) {
            counter++;

            if ((playing_field[i][j] == 1 && It_a_free_moveFRIEND(i, j,
playing_field, &blocking_counter)) || (playing_field[i][j] == 3 &&
It_a_free_moveQUEEN(i, j, playing_field, priority, &blocking_counter))) {

                Displaying_the_game_space_on_the_screenBOT(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, 1);
                Beep(300, 300);
                Sleep(500);



                if (It_is_possible_to_shoot_down(i, j, playing_field,
number_of_your_checkers)) {
                    Displaying_the_game_space_on_the_screenBOT(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, INT_MIN, INT_MIN, INT_MIN);
                    Sleep(500);
                    return 1;

                }
                else if (playing_field[i][j] == 3 &&
It_is_possible_to_shoot_downQUEEN(i, j, playing_field,
number_of_your_checkers)) {
                    Displaying_the_game_space_on_the_screenBOT(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, INT_MIN, INT_MIN, INT_MIN);
                    Sleep(500);
                    return  1;
```

```
                }

            }

        }

    }
    if (counter == *number_of_opponent_checkers) {
        return 0;
    }
}

}
int The_passage_of_the_bot_aimed_at_safe_passage(int playing_field[][8],
char* name_player, int* point, int priority, int*
number_of_opponent_checkers, int* number_of_your_checkers) {
int counter = 0;
int blocking_counter = 0;

for (int i = 0; i <= 7; i++) {
    for (int j = 0; j <= 7; j++) {

        if (playing_field[i][j] == 1 || playing_field[i][j] == 3) {
            counter++;

            if ((playing_field[i][j] == 1 && It_a_free_moveFRIEND(i, j,
playing_field, &blocking_counter)) || (playing_field[i][j] == 3 &&
It_a_free_moveQUEEN(i, j, playing_field, priority, &blocking_counter))) {
                Displaying_the_game_space_on_the_screenBOT(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, 1);
                Sleep(500);
                Beep(300, 300);



                if (There_is_a_safe_passage(i, j, playing_field)) {
                    Displaying_the_game_space_on_the_screenBOT(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, INT_MIN, INT_MIN, INT_MIN);
                    Sleep(500);
                    return 1;

                }





            }

        }
```

```
    }

    if (counter == *number_of_opponent_checkers) {
        return 0;
    }
}

}
void The_bots_passage_on_a_dangerous_move(int playing_field[][8], char*
name_player, int* point, int priority, int* number_of_opponent_checkers, int*
number_of_your_checkers) {
int blocking_counter = 0;



for (int i = 7; i >= 0; i -  - ) {
    for (int j = 7; j >= 0; j -  - ) {

        if (playing_field[i][j] == 1 || playing_field[i][j] == 3) {



            if ((playing_field[i][j] == 1 && It_a_free_moveFRIEND(i, j,
playing_field, &blocking_counter)) || (playing_field[i][j] == 3 &&
It_a_free_moveQUEEN(i, j, playing_field, priority, &blocking_counter))) {
                Displaying_the_game_space_on_the_screenBOT(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, i, j, 1);
                Sleep(500);
                Beep(300, 300);



                There_is_a_dangerous_move(i, j, playing_field);
                Displaying_the_game_space_on_the_screenBOT(playing_field,
name_player, point, priority, *number_of_opponent_checkers,
*number_of_your_checkers, INT_MIN, INT_MIN, INT_MIN);
                Sleep(500);

                return;
            }

        }




    }
```

```
}

}
void Playing_with_a_bot(int playing_field[][8], char* name_player, int*
point) {
int number_of_opponent_checkers = 12;
int number_of_your_checkers = 12;

int priority = 2;



Displaying_the_game_space_on_the_screenBOT(playing_field, name_player, point,
priority, number_of_opponent_checkers, number_of_your_checkers, INT_MIN,
INT_MIN, INT_MIN);



while (number_of_opponent_checkers != 0 && number_of_your_checkers != 0)
{
    if (priority == 2) {

        Players_turn_with_a_BOT(playing_field, name_player, point, priority,
&number_of_opponent_checkers, &number_of_your_checkers);
        priority = 3;

    }
    else if (priority == 3) {
        int flag = A_bot_pass_aimed_at_taking_checkers(playing_field,
name_player, point, priority, &number_of_opponent_checkers,
&number_of_your_checkers);
        if (flag == 0) {

            flag =
The_passage_of_the_bot_aimed_at_safe_passage(playing_field, name_player,
point, priority, &number_of_opponent_checkers, &number_of_your_checkers);
            if (flag == 0)
            {
                The_bots_passage_on_a_dangerous_move(playing_field,
name_player, point, priority, &number_of_opponent_checkers,
&number_of_your_checkers);

            }

        }



        priority = 2;
    }



}
```

```
if (number_of_opponent_checkers == 0) {

    (*point) += 5;
    Victory_screen();
}
else {
    (*point)  - = 5;
    Loss_screen();
}}
void Game(char* name_player, int* point) {
int playing_field[8][8] = { {0,1,0,1,0,1,0,1},
                            {1,0,1,0,1,0,1,0},
                            {0,1,0,1,0,1,0,1},
                            {0,0,0,0,0,0,0,0},
                            {0,0,0,0,0,0,0,0},
                            {2,0,2,0,2,0,2,0},
                            {0,2,0,2,0,2,0,2},
                            {2,0,2,0,2,0,2,0} };




if (Game_mode_selection() == 1) {

    A_game_for_two(playing_field, name_player, point);

}
else
{
    Playing_with_a_bot(playing_field, name_player, point);



}

}
```

Файл Graphic_elements.c

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <locale.h>
#include <windows.h>
#include <string.h>
#include"Checking.h"
void Notification_of_the_course_and_number_of_checkers(char* name_player, int
priority, int number_of_opponent_checkers, int number_of_your_checkers) {
if (priority == 2) {
    printf("\n\t\x1b[35mВаш ход! ( %s) \x1b[0m\n\tКоличество оставшихся
шашек: \x1b[35m%d\x1b[0m", name_player, number_of_your_checkers);
```

```
}
else if (priority == 1) {
    printf("\n\t\x1b[36mХод друга!  \x1b[0m\n\tКоличество оставшихся шашек:
\x1b[36m%d\x1b[0m", number_of_opponent_checkers);
}
else if (priority == 3) {
    printf("\n\t\x1b[36mХод Бота!  \x1b[0m\n\tКоличество оставшихся шашек:
\x1b[36m%d\x1b[0m", number_of_opponent_checkers);
}

}
void Drawing_arrows_and_choosing_a_checker(int flag) {if (flag == 1) {
printf("\n\t\x1b[32m Совершите ход! \x1b[0m");
    printf("\n\t_________________________________");
    printf("\n\t|              |              |");
    printf("\n\t|     < -  -  -      |       -  -  - >    |");
    printf("\n\t|_______________|_______________|");
    printf("\n\t\x1b[32m*ход выполняется нажатием стрелок\x1b[0m");
}
else if (flag == 0) {
    printf("\n\t\x1b[32mВыберите шашку! \x1b[0m");
    printf("\n\t_________________________________");
    printf("\n\t|              |              |");
    printf("\n\t|    ПРОБЕЛ    |     ENTER    |");
    printf("\n\t|_______________|_______________|");
    printf("\n\t\x1b[32m*нажмите пробел ,чтобы изменить выбор\x1b[0m");
    printf("\n\t\x1b[32m*нажмите ENTER ,чтобы подтвердить выбор\x1b[0m");
}
else if (flag == 2) {

    printf("\n\t\x1b[32m Выберите направление хода! \x1b[0m");
    printf("\n\t_________________________________");
    printf("\n\t|              |              |");
    printf("\n\t|       |      |       ^      |");
    printf("\n\t|       v      |       |      |");
    printf("\n\t|_______________|_______________|");
    printf("\n\t\x1b[32m*ход выполняется нажатием стрелок\x1b[0m");

}

}
void Displaying_the_game_space_on_the_screen(int playing_field[][8], char*
name_player, int* point, int priority, int number_of_opponent_checkers, int
number_of_your_checkers, int index_i, int index_j, int flag) {
system("cls");
printf("\t_____________________________\n");
printf("\t| \x1b[32m       ИГРА НА ДВОИХ  \x1b[0m     |\n");
printf("\t|____________________________|\n");
printf("\n\t\x1b[32mигрок:\x1b[0m %s\n\t\x1b[32mбаллы:\x1b[0m %d\n",
name_player, *point);
printf("\n\n\t\033[44m    a  b  c  d  e  f  g  h  \033[0m\n");for (int i = 0;
i < 8; i++) {
    for (int j = 0; j < 8; j++) {
```

```
        if (j == 0) {
            printf("\t\033[44m %d \033[0m", i + 1);

        }



        if (playing_field[i][j] == 0 && (j + i) % 2 == 0) {
            printf("\033[47m   \033[0m");

        }
        else if (priority == 2 && index_i == i && index_j == j &&
(playing_field[i][j] == 2 || playing_field[i][j] == 4)) {
            if (playing_field[i][j] == 2) {
                printf("\x1b[35m\033[42m o \033[0m\x1b[0m");
            }
            else {
                printf("\x1b[35m\033[42m O \033[0m\x1b[0m");
            }

        }
        else if (priority == 1 && index_i == i && index_j == j &&
(playing_field[i][j] == 1 || playing_field[i][j] == 3)) {
            if (playing_field[i][j] == 1) {
                printf("\x1b[36m\033[42m o \033[0m\x1b[0m");
            }
            else {
                printf("\x1b[36m\033[42m O \033[0m\x1b[0m");
            }

        }
        else if (playing_field[i][j] == 1) {
            printf("\x1b[36m o \x1b[0m");

        }
        else if (playing_field[i][j] == 2) {
            printf("\x1b[35m o \x1b[0m");

        }
        else if (playing_field[i][j] == 3) {
            printf("\x1b[36m O \x1b[0m");

        }
        else if (playing_field[i][j] == 4) {
            printf("\x1b[35m O \x1b[0m");

        }
        else {
            printf("   ");
        }
        if (j == 7) {
```

```
            printf("\033[44m \033[0m");
        }

    }
    printf("\n");

}
printf("\t\033[44m                            \033[0m\n");
Notification_of_the_course_and_number_of_checkers(name_player, priority,
number_of_opponent_checkers, number_of_your_checkers);
Drawing_arrows_and_choosing_a_checker(flag);}

void Displaying_the_game_space_on_the_screenBOT(int playing_field[][8], char*
name_player, int* point, int priority, int number_of_opponent_checkers, int
number_of_your_checkers, int index_i, int index_j, int flag) {system("cls");
printf("\t_______________________________\n");
printf("\t| \x1b[32m       ИГРА С  БОТОМ  \x1b[0m     |\n");
printf("\t|_____________________________|\n");
printf("\n\t\x1b[32mигрок:\x1b[0m %s\n\t\x1b[32mбаллы:\x1b[0m %d\n",
name_player, *point);
printf("\n\n\t\033[44m    a  b  c  d  e  f  g  h  \033[0m\n");for (int i = 0;
i < 8; i++) {    for (int j = 0; j < 8; j++) {

        if (j == 0) {
            printf("\t\033[44m %d \033[0m", i + 1);
        }



        if (playing_field[i][j] == 0 && (j + i) % 2 == 0) {
            printf("\033[47m   \033[0m");

        }
        else if (priority == 2 && index_i == i && index_j == j &&
(playing_field[i][j] == 2 || playing_field[i][j] == 4)) {
            if (playing_field[i][j] == 2) {
                printf("\x1b[35m\033[42m o \033[0m\x1b[0m");
            }
            else {
                printf("\x1b[35m\033[42m O \033[0m\x1b[0m");
            }

        }
        else if (priority == 1 && index_i == i && index_j == j &&
(playing_field[i][j] == 1 || playing_field[i][j] == 3)) {
            if (playing_field[i][j] == 1) {
                printf("\x1b[36m\033[42m o \033[0m\x1b[0m");
            }
            else {
                printf("\x1b[36m\033[42m O \033[0m\x1b[0m");
            }

        }
```

```
        else if (playing_field[i][j] == 1) {
            printf("\x1b[36m o \x1b[0m");

        }
        else if (playing_field[i][j] == 2) {
            printf("\x1b[35m o \x1b[0m");

        }
        else if (playing_field[i][j] == 3) {
            printf("\x1b[36m O \x1b[0m");

        }
        else if (playing_field[i][j] == 4) {
            printf("\x1b[35m O \x1b[0m");

        }
        else {
            printf("   ");
        }
        if (j == 7) {
            printf("\033[44m \033[0m");
        }

    }
    printf("\n");

}
printf("\t\033[44m                            \033[0m\n");
Notification_of_the_course_and_number_of_checkers(name_player, priority,
number_of_opponent_checkers, number_of_your_checkers);
Drawing_arrows_and_choosing_a_checker(flag);
 }
void Victory_screen() {
system("cls");

printf("\t_______________________________\n");
printf("\t|      \x1b[32m      ПОБЕДА!!!  \x1b[0m       |\n");
printf("\t|____________________________|\n");

printf("\n\n\n\n\t\x1b[35mВЫ ПОБЕДИЛИ!!! +5 баллов :)\x1b[0m\n");

Sleep(1500);

}
void Loss_screen() {
system("cls");

printf("\t_______________________________\n");
printf("\t|     \x1b[31m     ПРОИГРЫШ :(  \x1b[0m     |\n");
printf("\t|____________________________|\n");
```

```
printf("\n\n\n\n\t\x1b[36mВЫ ПРОИГРАЛИ...  - 5 баллов :(\x1b[0m\n");

Sleep(1500);

}
```